Cloud Computing Lab — Experiment 5

Implementation of Various Scheduling Mechanisms Using an Open-Source Cloud Simulator (CloudSim Plus)

Field	Details
Name	TODO: Add your name
Roll Number	TODO: Add your roll number
Section	BTECH_CSE_SECB_2023
Course	Cloud Computing Lab (CSE12045)
University	Adamas University (CSE, SOET)
Date of completion	11 November 2026

1. Aim

Set up a Java 17 + Maven environment in the Ubuntu VM, create a **CloudSim Plus** project, and implement and run a simulation of the four combinations of VM-level and cloudlet-level scheduling policies:

Scenario	VM scheduler (host → VMs)	Cloudlet scheduler (VM → cloudlets)
A	`VmSchedulerSpaceShared`	`CloudletSchedulerSpaceShared`
B	`VmSchedulerSpaceShared`	`CloudletSchedulerTimeShared`
C	`VmSchedulerTimeShared`	`CloudletSchedulerSpaceShared`
D	`VmSchedulerTimeShared`	`CloudletSchedulerTimeShared`

2. Theory

CloudSim Plus is an open-source Java framework that simulates a cloud data centre: hosts (physical machines), virtual machines (VMs), cloudlets (the workloads, i.e. jobs) and a broker that submits them. Because it only simulates the hardware, resource-management policies can be tested quickly and repeatably without a real cloud.

Scheduling happens at **two levels**:

Level	Class	Decides
Host → VMs	`VmScheduler`	How the host's processing elements (PEs, i.e. CPU cores) are given to the VMs placed on it
VM → cloudlets	`CloudletScheduler`	How a VM's PEs are given to the cloudlets submitted to it

Policy	Meaning
Space-shared	Each PE is dedicated to one VM (or cloudlet) at a time. Others have to wait until it finishes, so work runs one after another when PEs are scarce.
Time-shared	PEs are shared. All VMs (or cloudlets) run at the same time, each getting a slice of the capacity, so each one runs slower but none has to wait.

Key parameters used by this experiment:

Item	Value
Host	1 host, 4 PEs × 1000 MIPS, 8192 MB RAM, 100000 bandwidth, 1000000 storage
VMs	2 VMs, each 2 PEs × 1000 MIPS, 2048 MB RAM, 10000 bandwidth
Cloudlets	4 cloudlets, 20000 MI each, 1 PE each

A cloudlet of 20000 million instructions on a 1000 MIPS PE needs 20000 / 1000 = **20 s** of CPU time. The simulator adds about 0.1 s of start-up delay, so an undisturbed cloudlet runs from 0.10 s to 20.10 s.

3. Requirements

Item	Value
Host	Windows 11, Oracle VirtualBox 7.2.18
VM	`Ubuntu-Cloud-Lab`: Ubuntu 26.04.1 LTS, kernel 7.0.0-31-generic, 4 vCPU, 6144 MB RAM, 25 GB disk
Java	OpenJDK 17.0.21.0.3 (`java` and `javac`)
Build tool	Apache Maven 3.9.12 (running on Java 17)
Version control	Git 2.53.0
Editor	GNU nano 8.7.1
Library	CloudSim Plus **7.3.0** (see section 5 for why not 8.5.5 as written in the manual)
Maven plugins	clean 3.2.0, resources 3.3.1, compiler 3.13.0 (`release 17`), dependency 3.7.0, exec 3.7.0

4. Procedure

All commands were run in a terminal in the VM. Screenshot numbers refer to section 7.

1. **Update Ubuntu** (Figures 1–3)

   ```
   sudo apt update
   sudo apt upgrade -y
   ```

   `apt update` refreshes the package index (225 packages upgradable). `apt upgrade` installed 221 packages; see section 5 (issue 4) for the one package that could not be downloaded.

2. **Install Java 17, Maven and Git, and check them** (Figures 4–5)

   ```
   sudo apt install openjdk-17-jdk maven git -y
   java -version
   javac -version
   mvn -version
   git --version
   ```

   Java and javac report 17.0.21.0.3, and Maven reports that it is using Java 17.0.21.0.3, so no version mismatch had to be resolved.

3. **Create the Maven project** (Figures 6–8)

   ```
   mkdir -p ~/CloudSchedulingLab/src/main/java/lab
   cd ~/CloudSchedulingLab
   nano pom.xml
   ```

   `pom.xml` was typed in `nano` exactly as in the manual and saved (Ctrl+O, Enter, Ctrl+X). It sets Java 17 as the compiler release and declares the CloudSim Plus dependency.

4. **Write the experiment** (Figures 9–14)

   ```
   nano src/main/java/lab/SchedulingExperiment.java
   ```

   The 122-line program from the manual was typed in `nano`. Its SHA-256 checksum was compared with the manual's text to confirm that nothing was mistyped. It builds one host, two VMs and four cloudlets, and runs scenarios A–D by passing a different `VmScheduler` and `CloudletScheduler` to `runScenario(...)`, printing each cloudlet's VM, status, start time and finish time.

5. **Resolve dependencies and clean** (Figures 15–16, 23)

   ```
   mvn dependency:resolve
   mvn clean
   ```

6. **Compile and run** (Figures 17–34)

   ```
   mvn clean compile
   mvn exec:java -Dexec.mainClass="lab.SchedulingExperiment" -Dexec.classpathScope=compile
   ```

   The `exec:` plugin prefix resolved by itself (exec-maven-plugin 3.7.0), so the fully qualified fallback command in the manual was not needed.

   As written, the manual's code and `pom.xml` do not compile and run. They needed three small fixes, described in the next section. The commands above are unchanged; only the files changed.

5. Problems found in the manual and how they were fixed

The manual was followed exactly first, and each failure was kept as a screenshot before being fixed. The original files are in `original-from-manual/`, the final ones in `project/`, and each fix is a patch in `patches/`.

#	Problem (what happened)	Root cause	Fix	Evidence
1	`mvn clean compile` fails: `package org.cloudbus.cloudsim.core does not exist`. Rewriting only the imports to `org.cloudsimplus.*` gives new errors: `CloudSim is not public` and `cannot find symbol getExecStartTime()`.	The code was written for CloudSim Plus 7.x. From 8.0.0 the library moved to the `org.cloudsimplus` packages and added `CloudSimPlus` (the old `CloudSim` class is no longer public), and by 8.5.5 `Cloudlet.getExecStartTime()` had been replaced by `getStartTime()` (checked by listing the 7.3.0, 8.0.0 and 8.5.5 jars). The manual asks for 8.5.5, which cannot compile this code.	In `pom.xml` the dependency version was changed **8.5.5 → 7.3.0** (the only 7.x release on Maven Central, built for Java 17). The Java source stayed byte-for-byte as in the manual at this point.	Figures 17–22; `patches/01-pom-cloudsimplus-version.diff`
2	The program starts but stops at the first scenario with `IllegalArgumentException: CloudletScheduler already has a Vm assigned to it. Each Vm must have its own CloudletScheduler instance.`	`createVms()` gives the **same** `CloudletScheduler` object to both VMs, which CloudSim Plus does not allow.	A factory (`Supplier<CloudletScheduler>`, e.g. `CloudletSchedulerSpaceShared::new`) is passed instead of a single object, and `createVms()` calls `scheduler.get()` once per VM. All class and method names are unchanged.	Figures 25–27; `patches/02-cloudlet-scheduler-per-vm.diff`
3	After fix 2 the program runs and reports 4/4 cloudlets, but in every scenario cloudlets 2 and 3 "finish" at the impossible time **9223372076.74**, and the log has warnings such as `requested 10000 Mbps of Bandwidth but no amount is available`.	`UtilizationModelFull` is applied to the CPU, **RAM and bandwidth**, so every cloudlet claims 100% of its VM's 2048 MB and 10000 bandwidth. Two cloudlets on one VM cannot both have that, so the second is stalled. The identical result in A–D shows it has nothing to do with the schedulers.	Each cloudlet's RAM and bandwidth utilization is set to 25% with `UtilizationModelDynamic(0.25)`. CPU utilization is still `UtilizationModelFull`, as in the manual.	Figures 28–34; `patches/03-cloudlet-ram-bw-utilization.diff`
4	`sudo apt upgrade -y` stops with `Failed to fetch ... openvpn_2.7.0-1ubuntu1.4_amd64.deb 403 Forbidden` from the Ubuntu mirrors.	The network the VM is on refuses that one file (the same error came from two different mirrors and on retries). It is unrelated to the lab.	`sudo apt upgrade -y --fix-missing` skipped it; the other 221 packages were upgraded. `openvpn` (not used in this lab) and four packages that Ubuntu is rolling out in phases are still upgradable.	Figures 2–3

Run 1 below is the manual's program with only fixes 1 and 2 applied. **Run 2** also has fix 3 and gives meaningful numbers, so it is the result used in the conclusions.

6. Output

Run 2 (fixes 1, 2 and 3): final result

All four scenarios give the same table (Figures 33–34). Cloudlets are listed in the order they finished.

Cloudlet	VM	Status	Start	Finish
0	0	SUCCESS	0.10	20.10
2	0	SUCCESS	0.10	20.10
1	1	SUCCESS	0.10	20.10
3	1	SUCCESS	0.10	20.10

`Completed Cloudlets: 4 / 4` was printed for each of A, B, C and D, with no warnings. The simulation ended at time 20.21 and Maven reported `BUILD SUCCESS`.

Run 1 (fixes 1 and 2 only): the manual's own logic

In all four scenarios (Figure 29):

Cloudlet	VM	Status	Start	Finish
0	0	SUCCESS	0.10	20.10
1	1	SUCCESS	0.10	20.10
2	0	SUCCESS	0.10	9223372076.74
3	1	SUCCESS	0.10	9223372076.74

The simulation "finished" at 9223372076.95 and the warnings in Figure 30 explain why: the second cloudlet on each VM was waiting for 2048 MB of RAM and 10000 of bandwidth that the first cloudlet was holding. (The size of the number, close to Java's `Long.MAX_VALUE` ÷ 10^9, suggests an overflow or placeholder time rather than a real wait. This is an inference, not something that was checked in the library's source.)

7. Screenshots

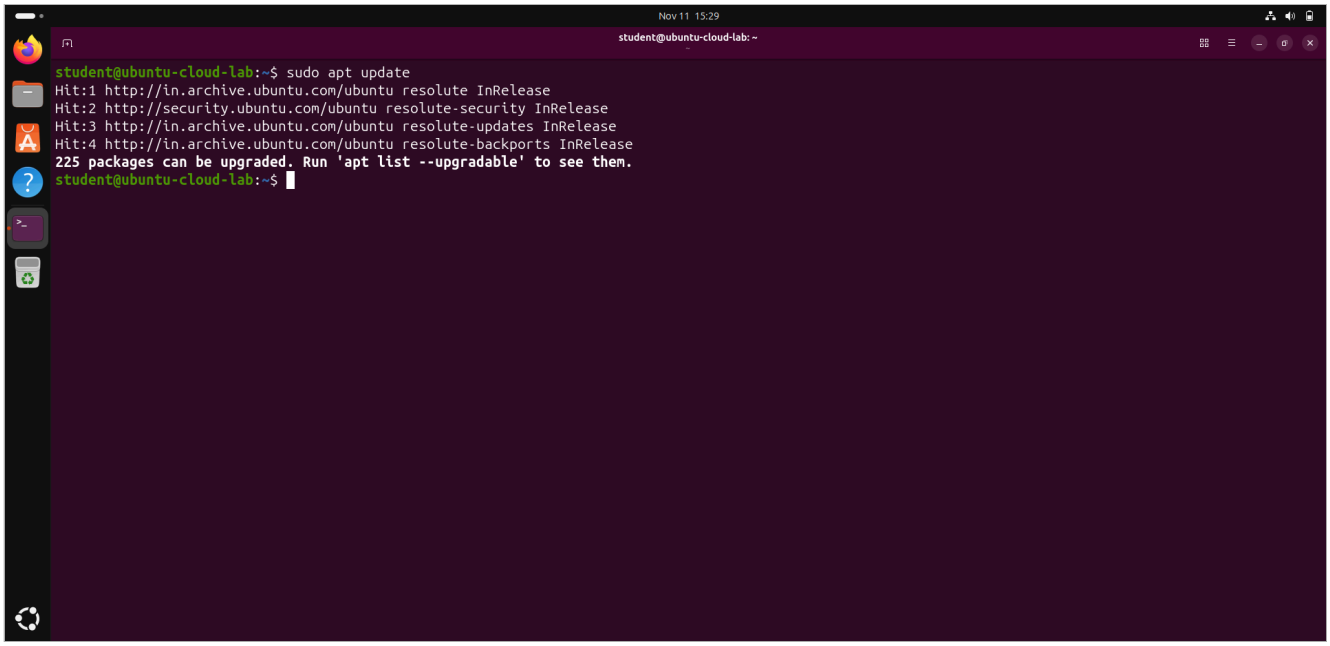


Figure 1: `sudo apt update`

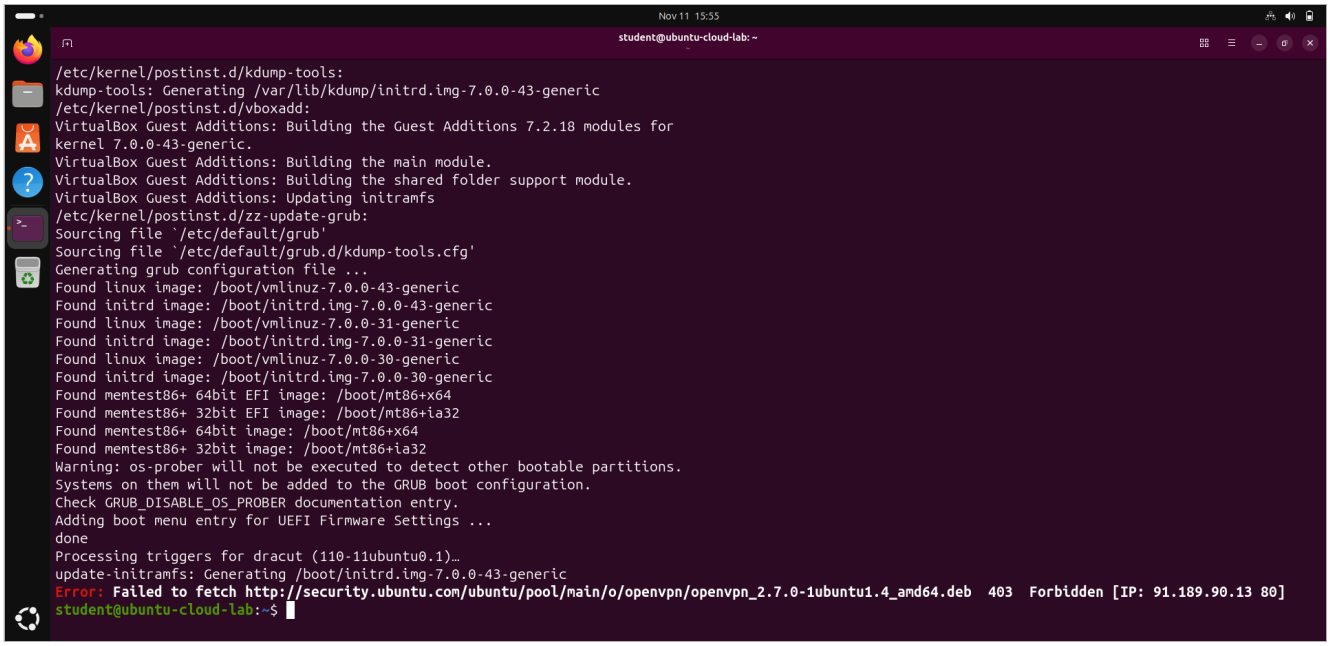


Figure 2: `sudo apt upgrade -y --fix-missing` finishing; only `openvpn` could not be fetched (403)

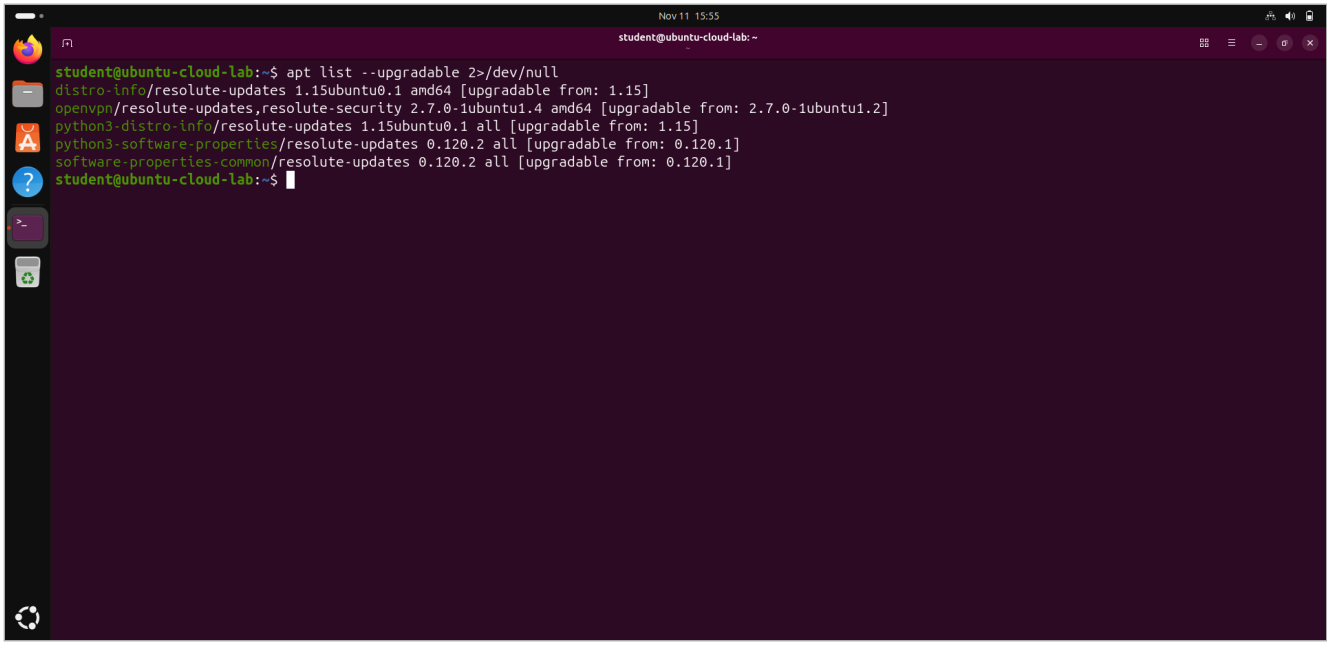


Figure 3: `apt list --upgradable`: the five packages still pending

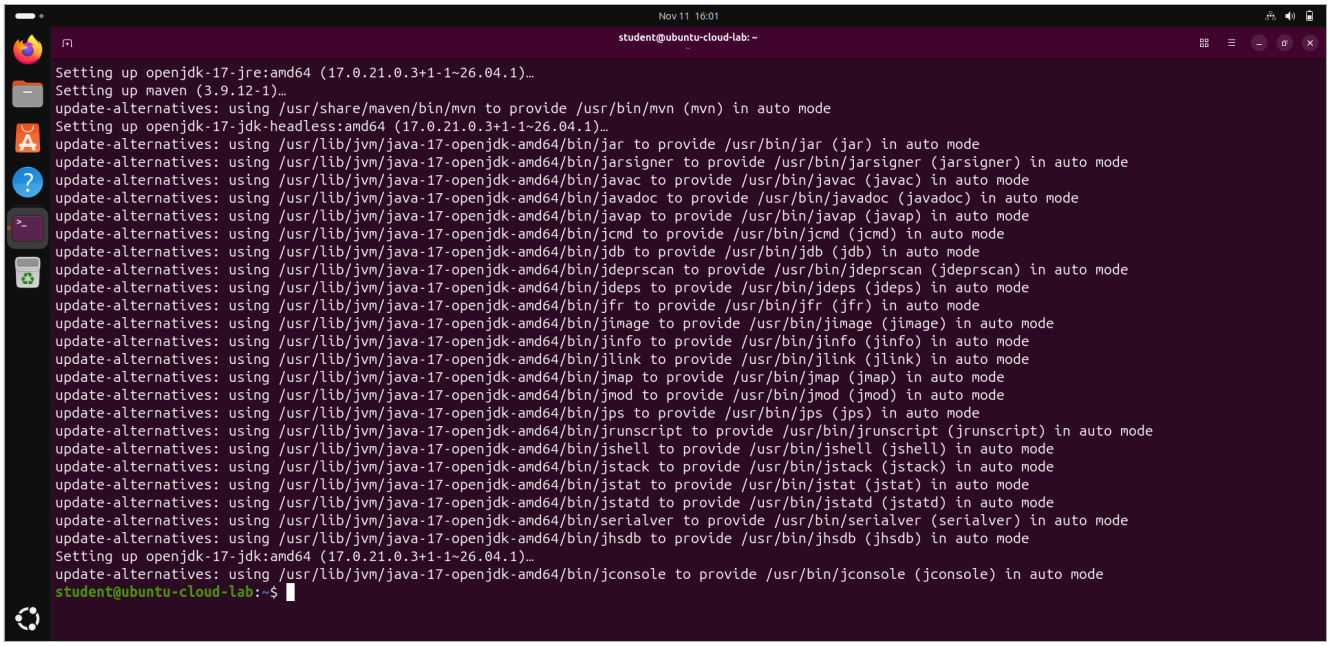


Figure 4: `sudo apt install openjdk-17-jdk maven git -y` finished

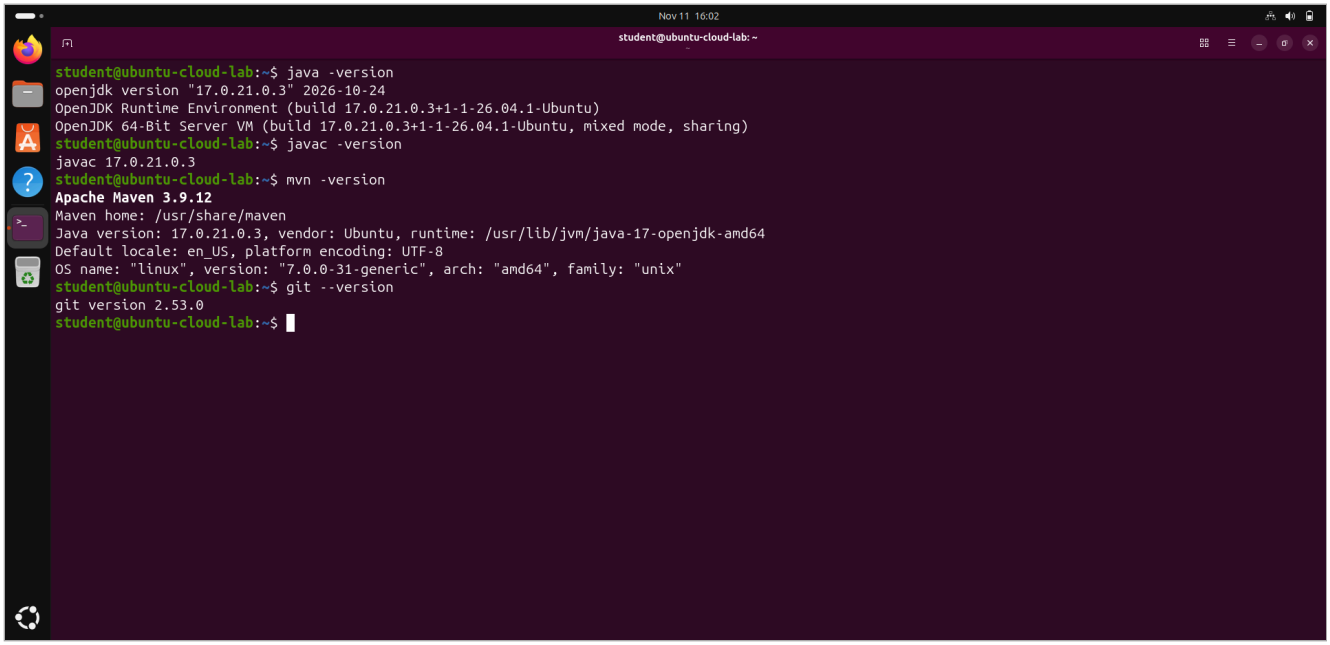


Figure 5: `java -version`, `javac -version`, `mvn -version`, `git --version`

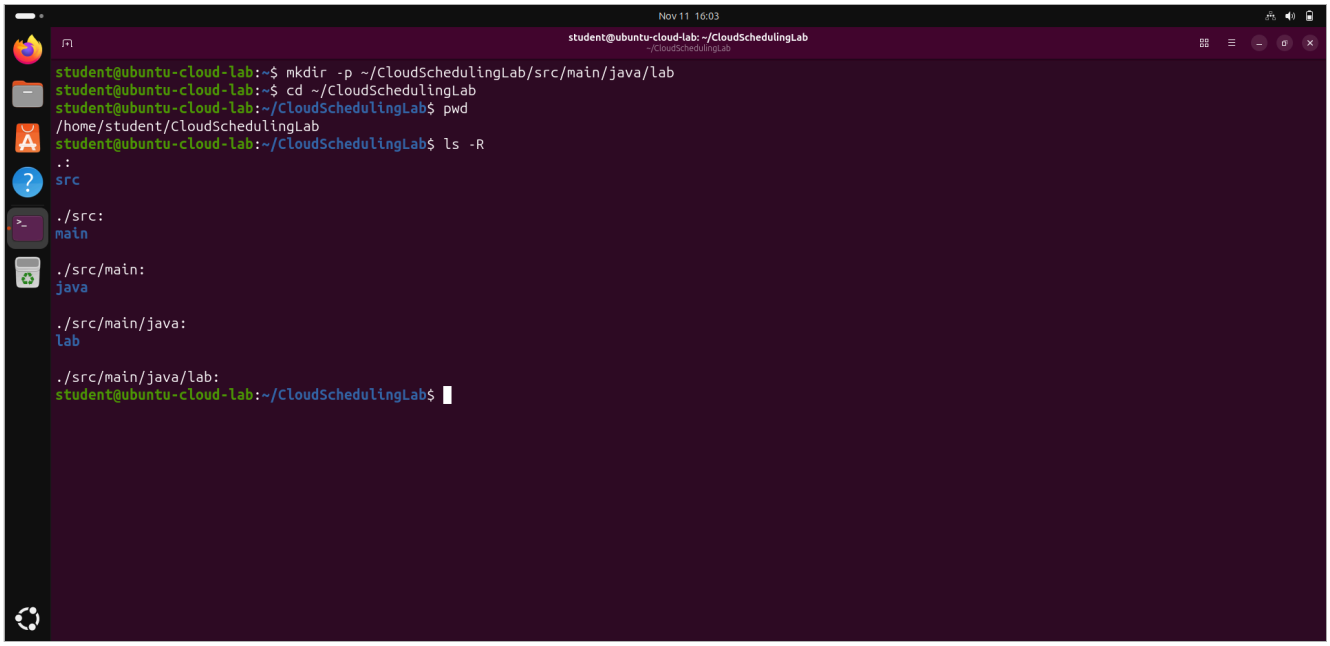


Figure 6: Project folders created and `ls -R`

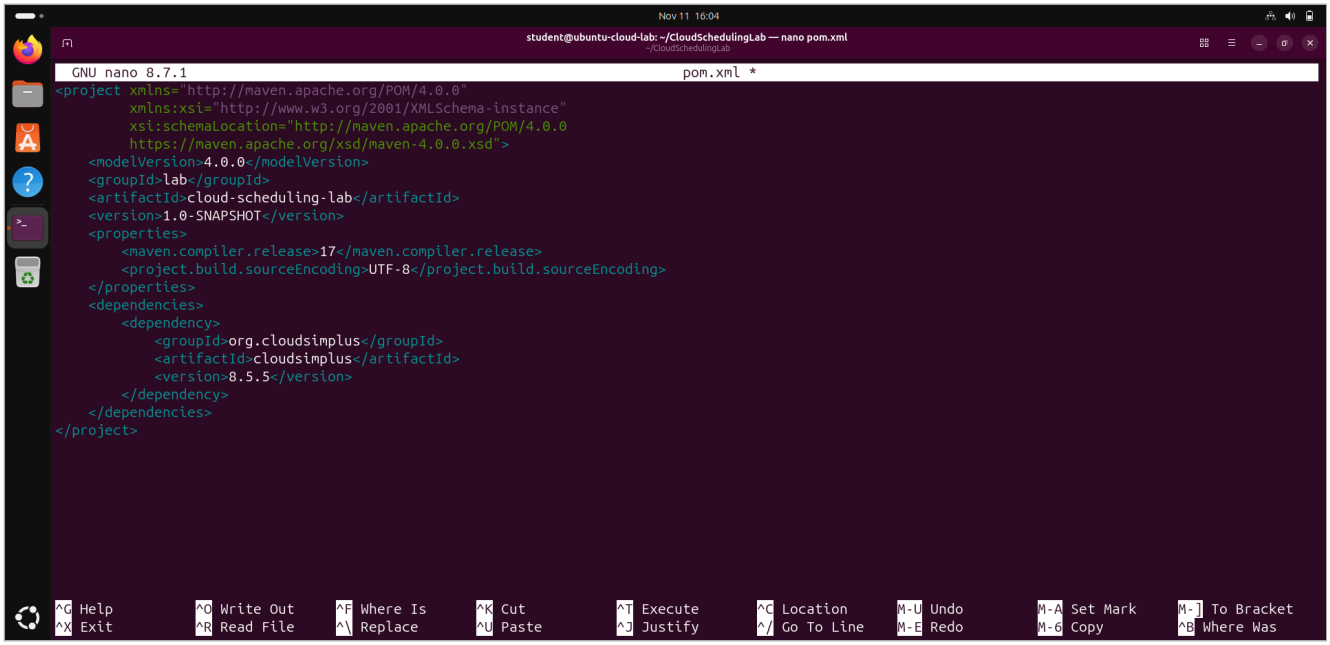


Figure 7: `pom.xml` typed in nano

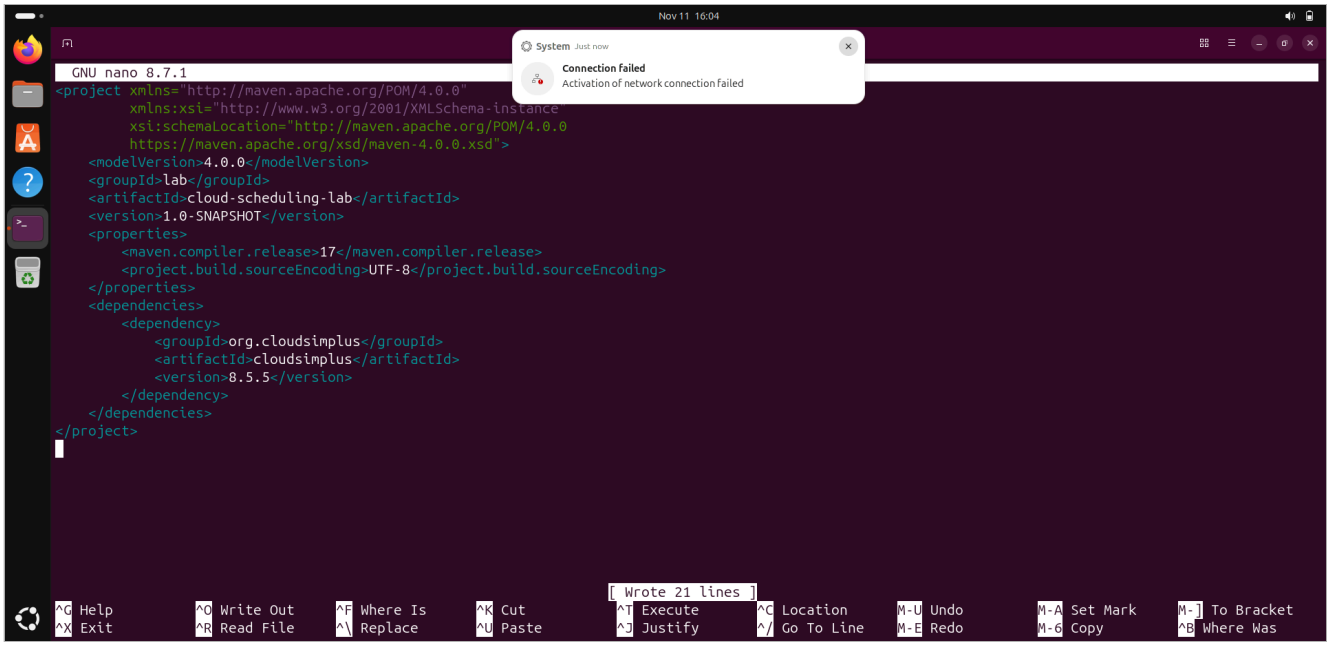


Figure 8: `pom.xml` saved (`Wrote 21 lines`)

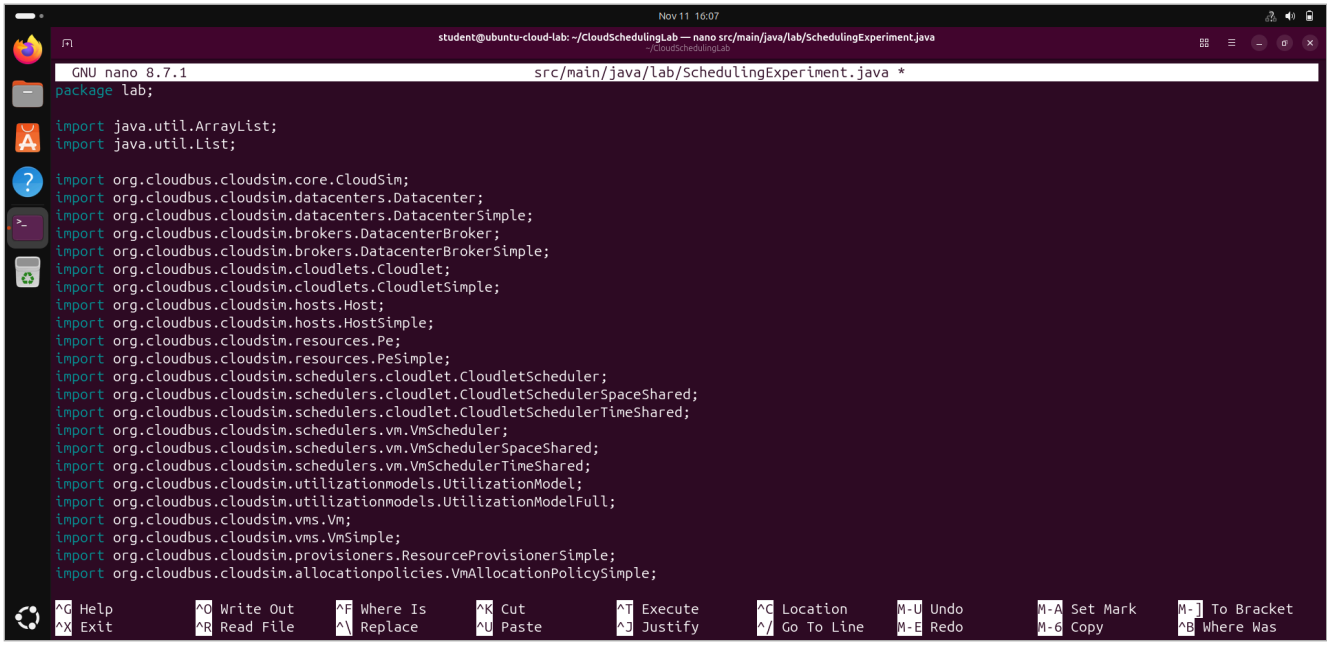


Figure 9: `SchedulingExperiment.java` in nano (page 1 of 5: imports)

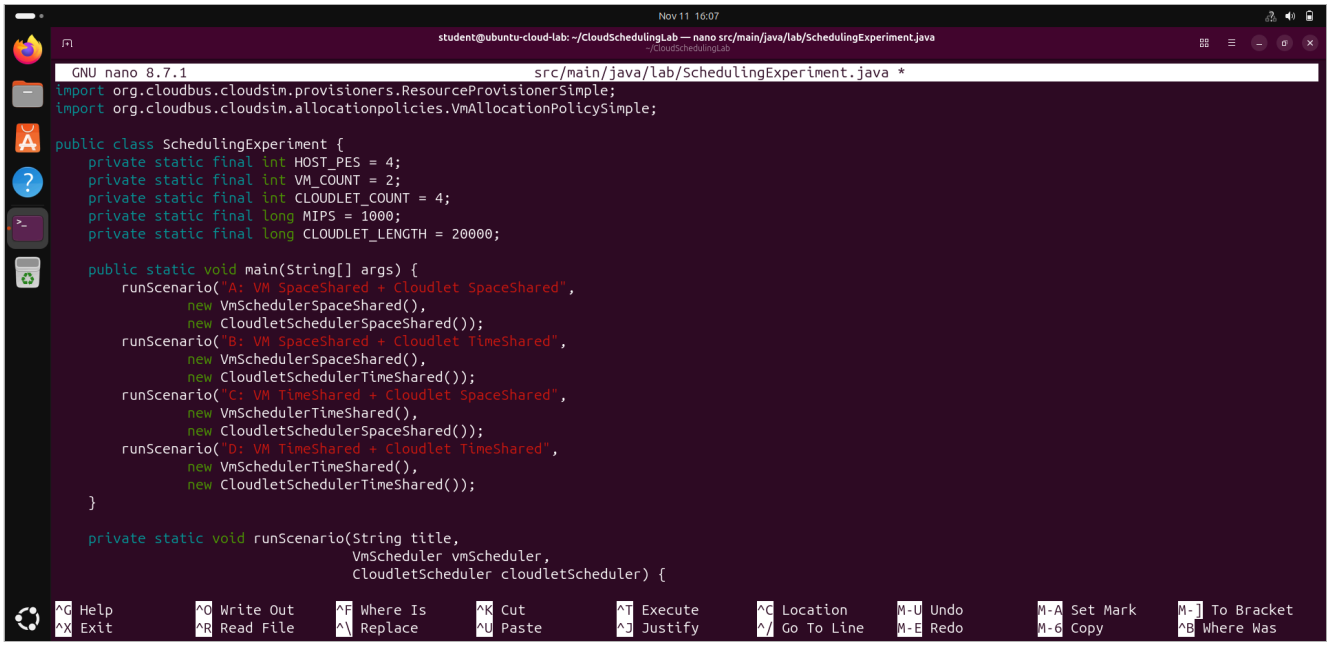


Figure 10: `SchedulingExperiment.java` (page 2 of 5)

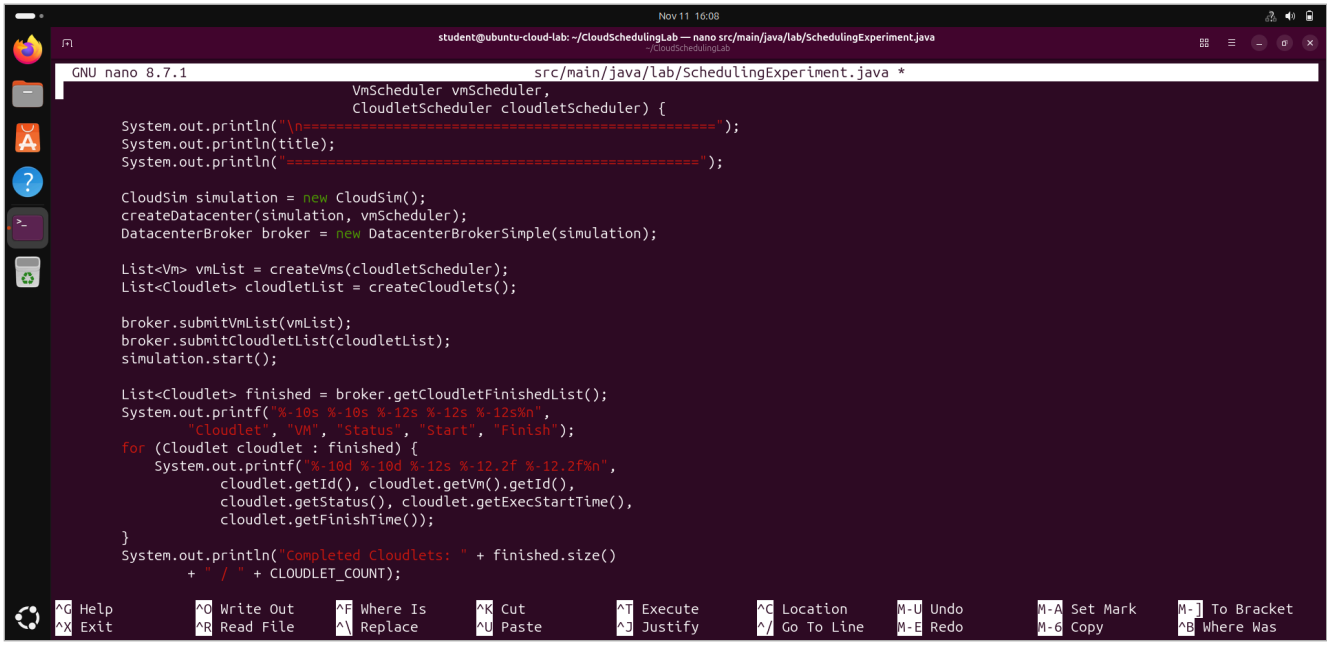


Figure 11: `SchedulingExperiment.java` (page 3 of 5: `runScenario`)

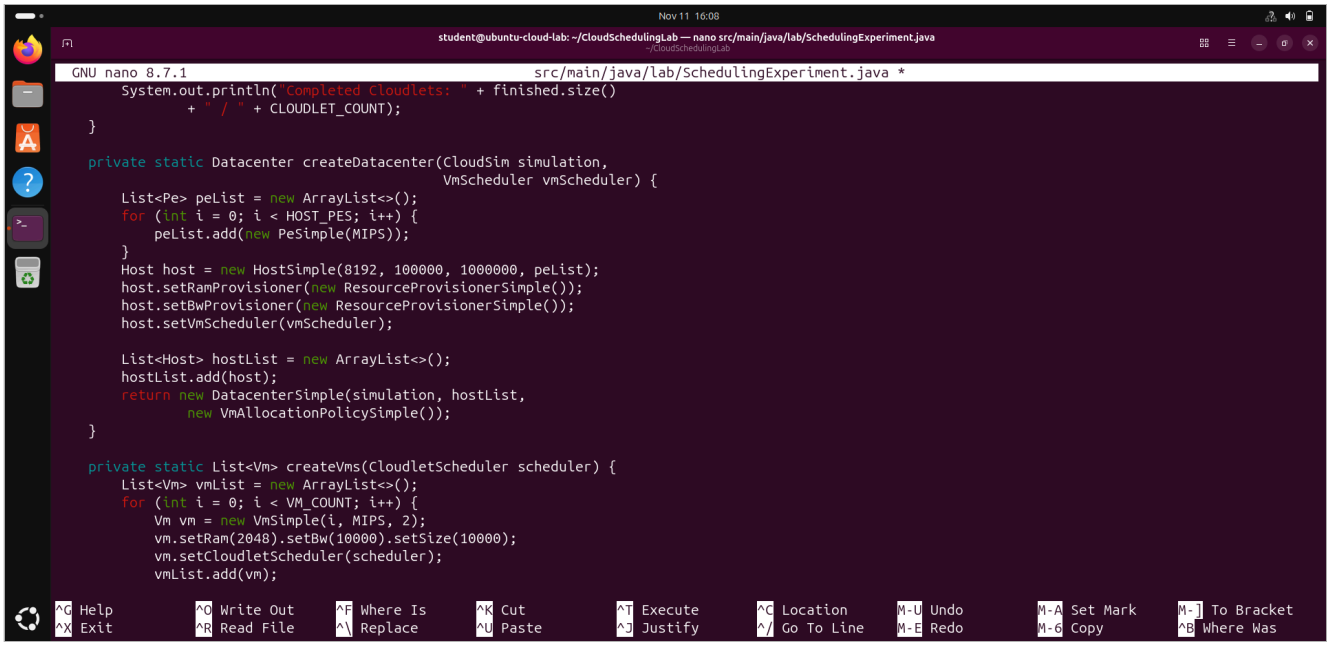


Figure 12: `SchedulingExperiment.java` (page 4 of 5: `createDatacenter`)

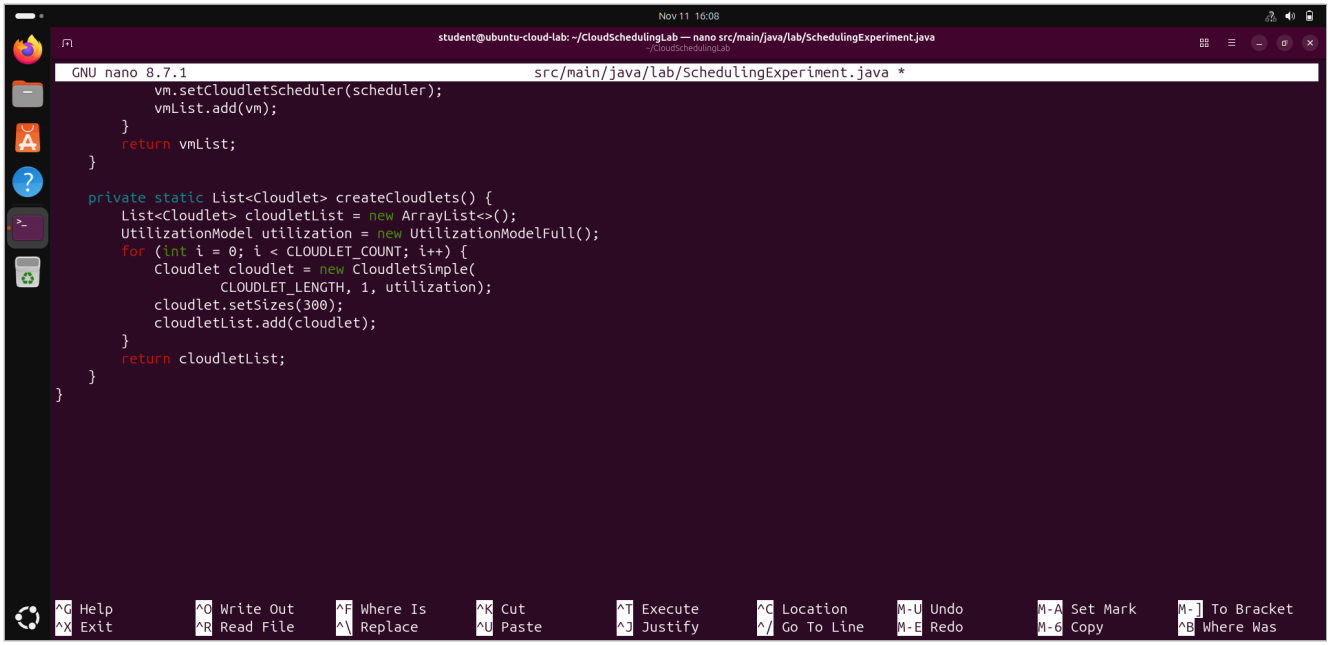


Figure 13: `SchedulingExperiment.java` (page 5 of 5: `createVms`, `createCloudlets`)

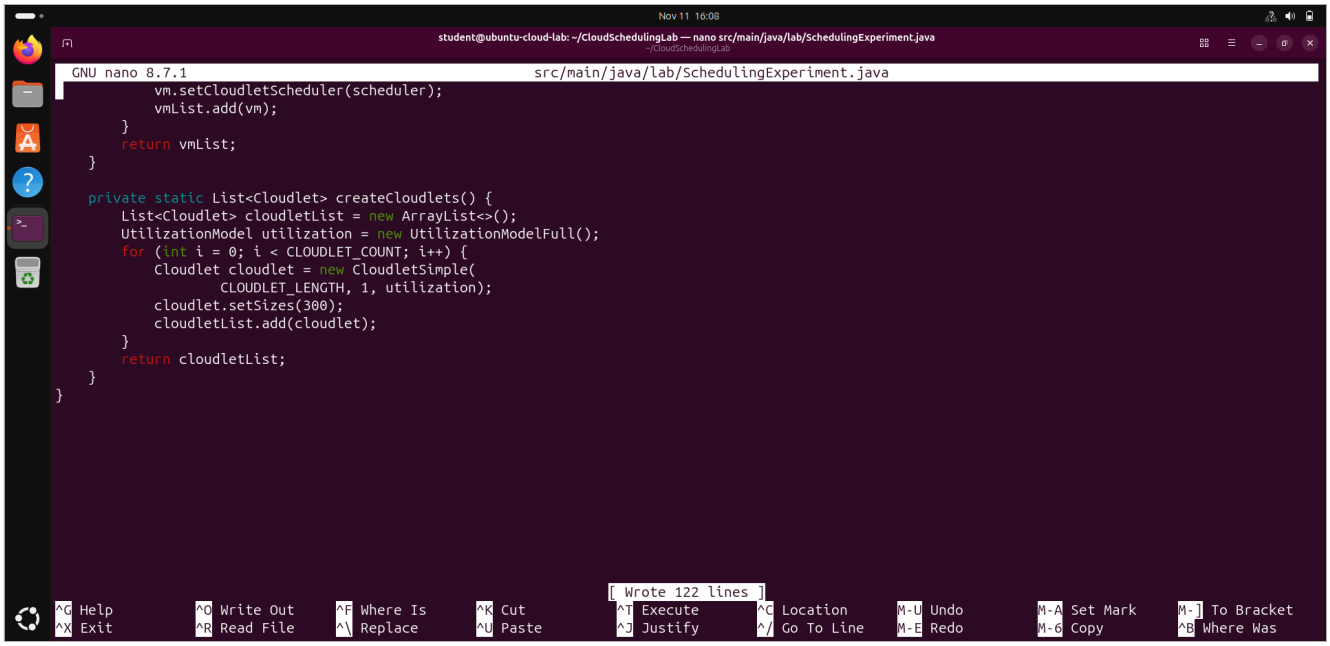


Figure 14: `SchedulingExperiment.java` saved (`Wrote 122 lines`)

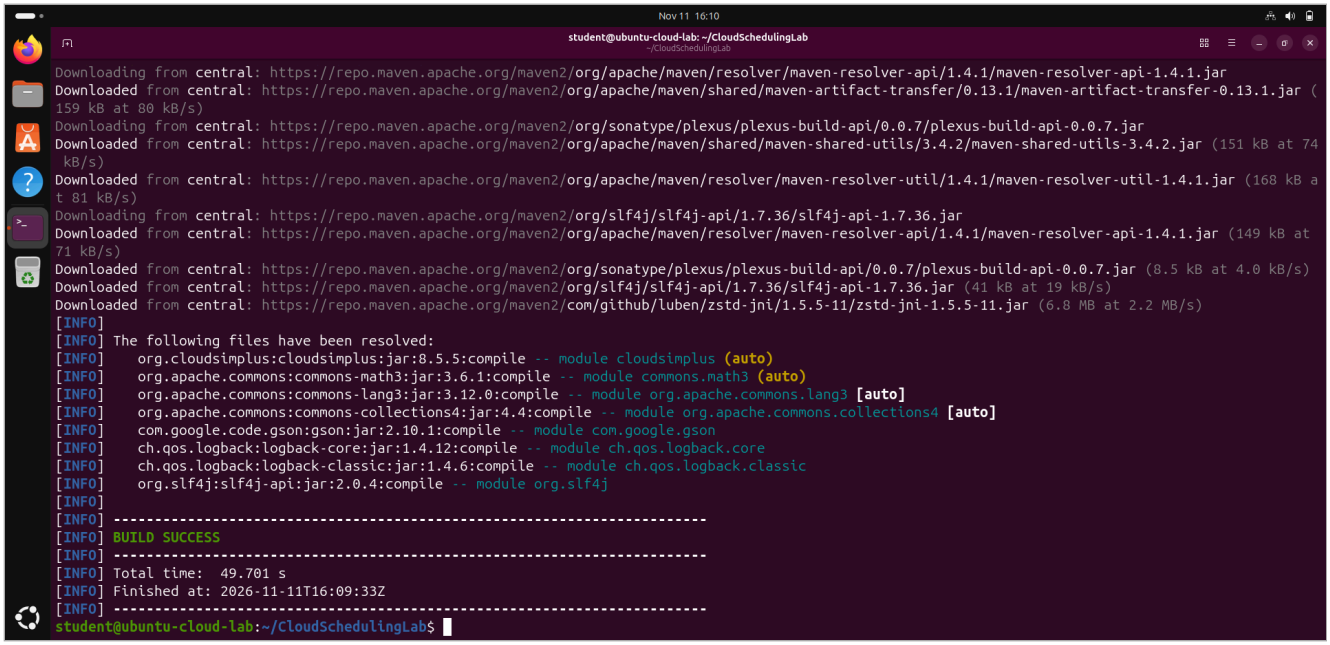


Figure 15: `mvn dependency:resolve` with CloudSim Plus 8.5.5 (as in the manual): BUILD SUCCESS

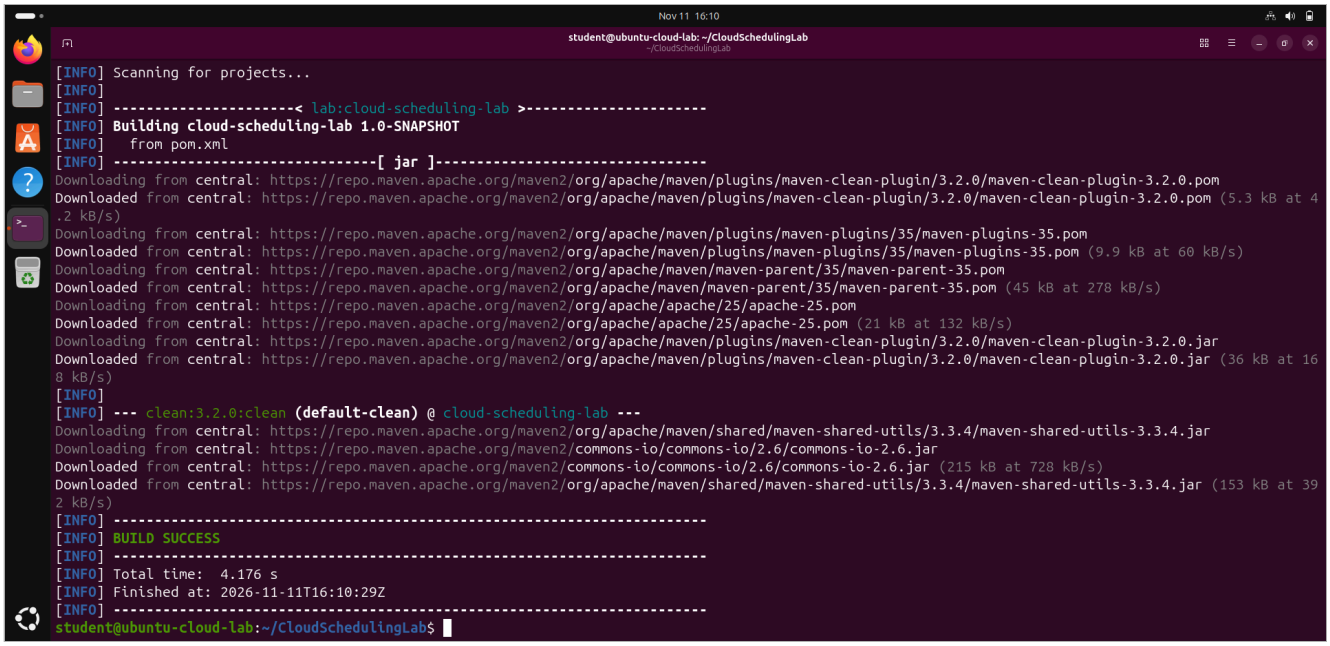


Figure 16: `mvn clean`: BUILD SUCCESS

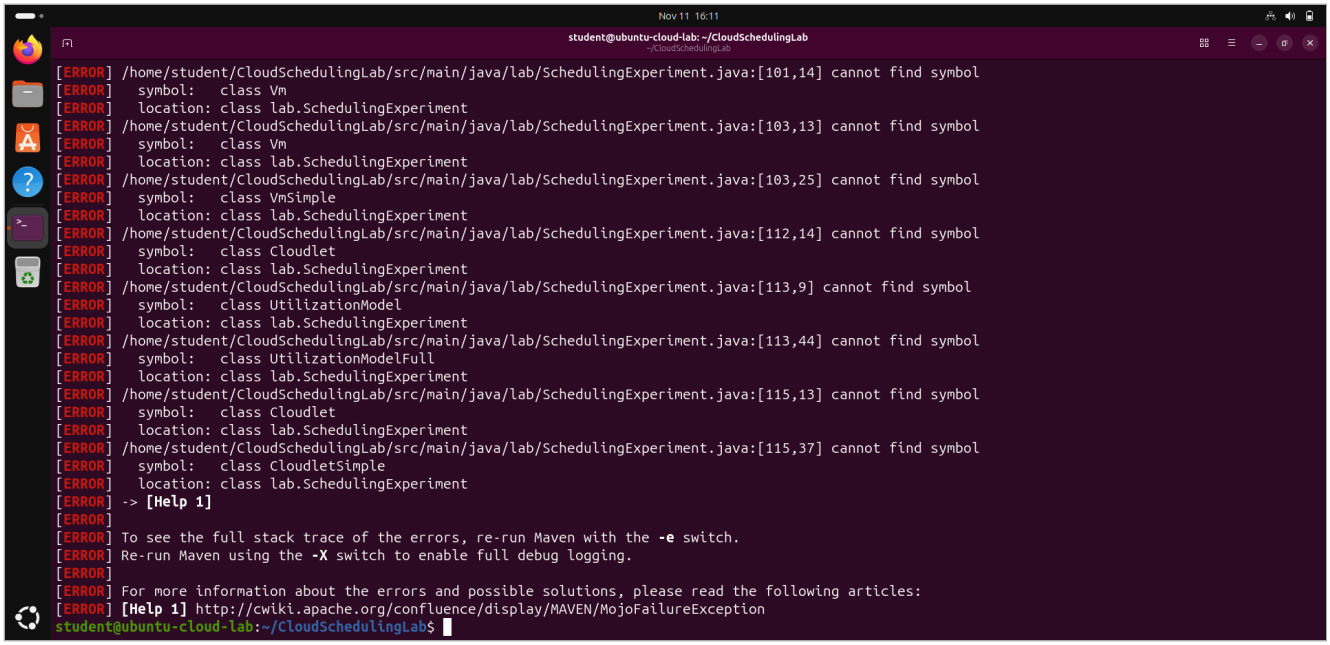


Figure 17: `mvn clean compile` with 8.5.5: BUILD FAILURE

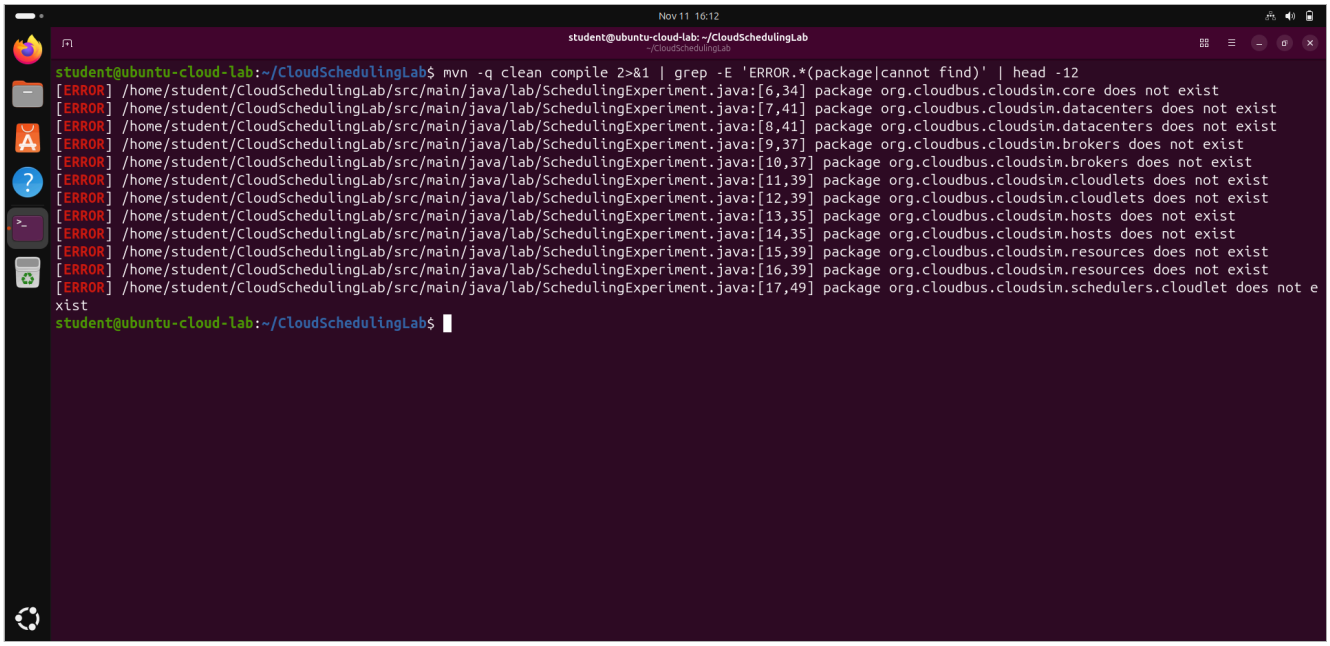


Figure 18: First compile errors: `package org.cloudbus.cloudsim.core does not exist`

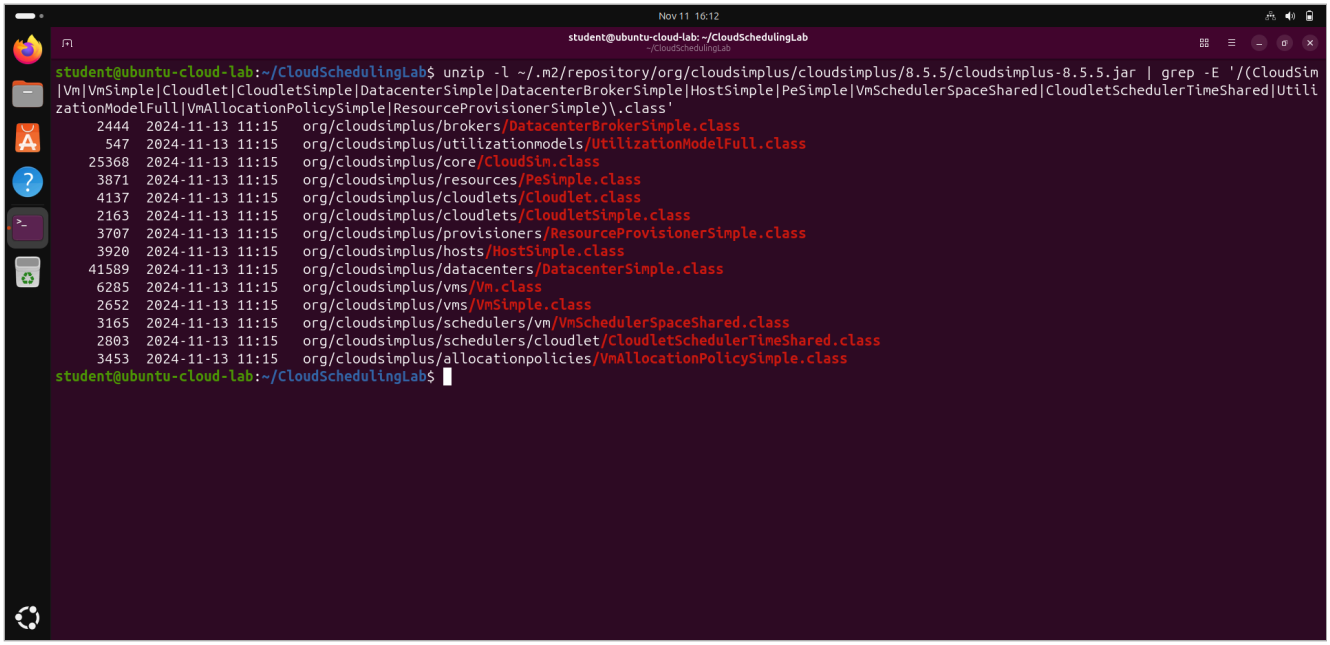


Figure 19: Inside the 8.5.5 jar the classes are in `org/cloudsimplus/...`

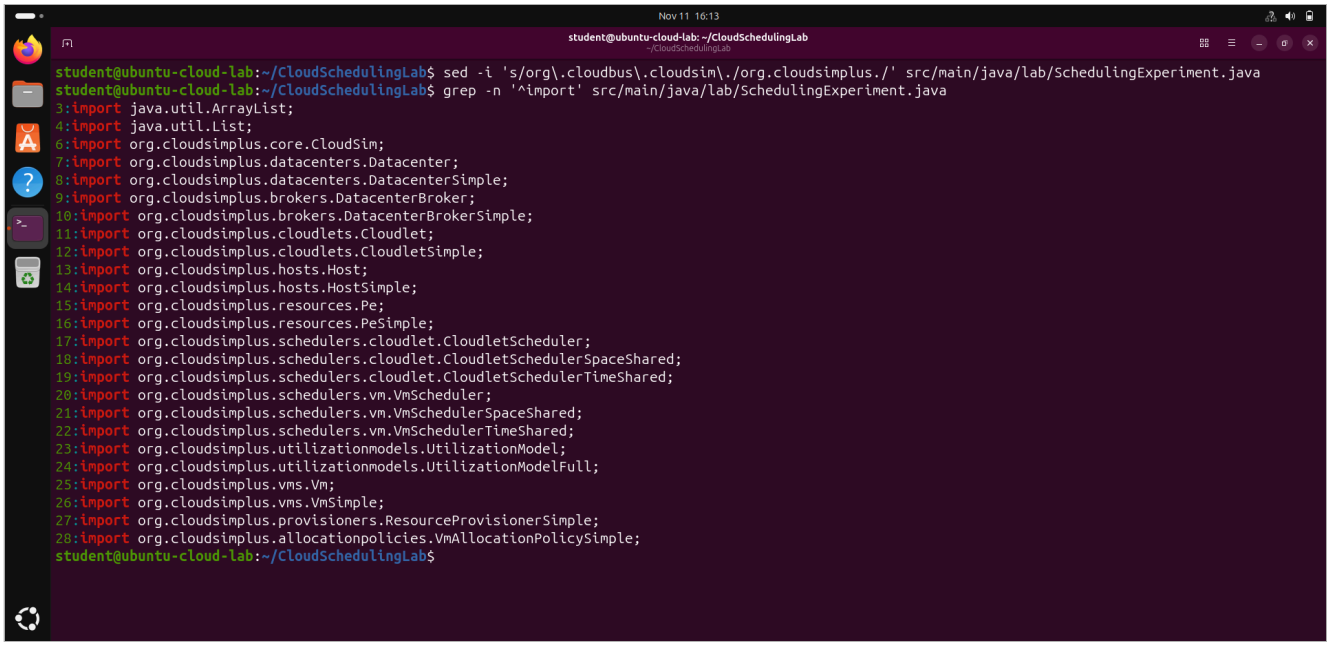


Figure 20: Trial: imports rewritten to `org.cloudsimplus.*`

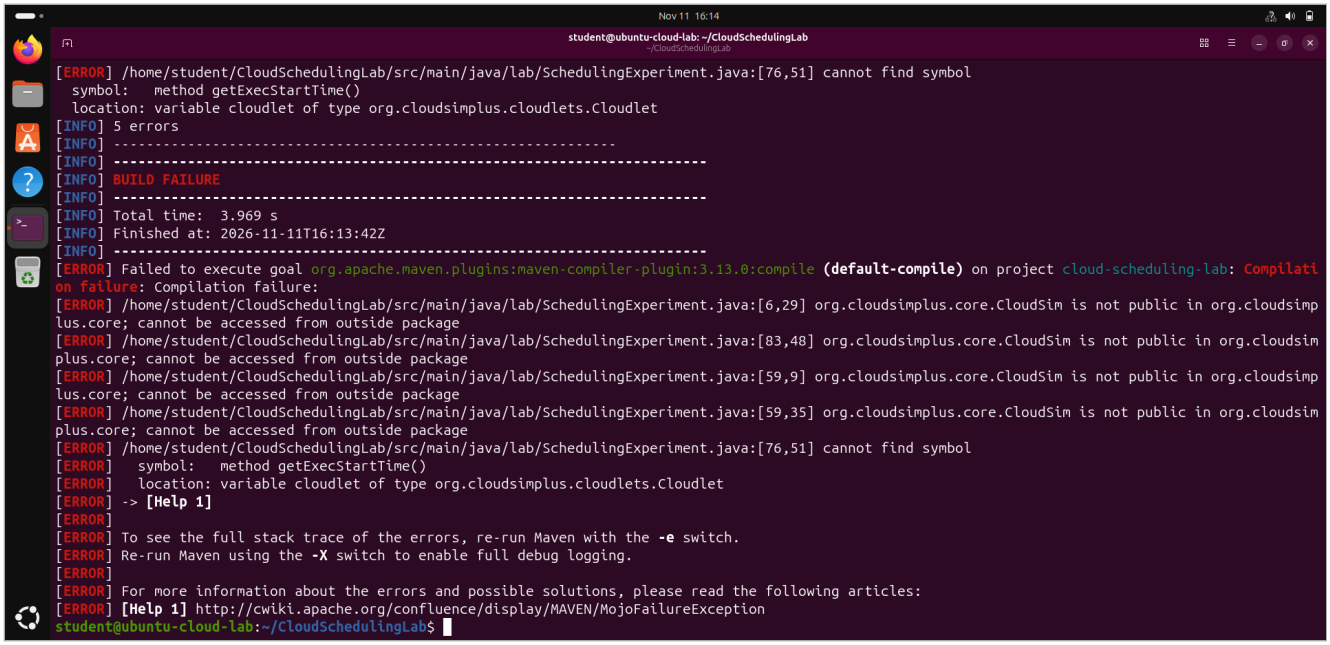


Figure 21: Still failing: `CloudSim is not public`, no `getExecStartTime()`

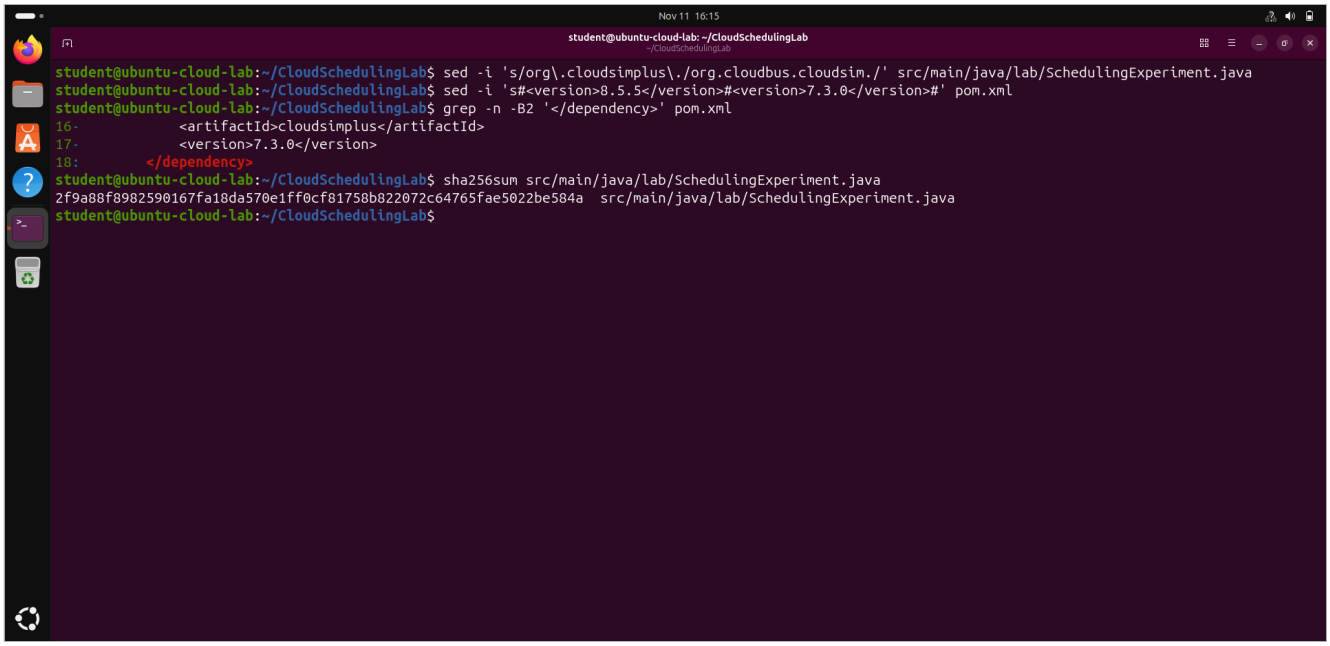


Figure 22: Fix 1: imports restored (checksum equals the manual's) and the dependency pinned to 7.3.0

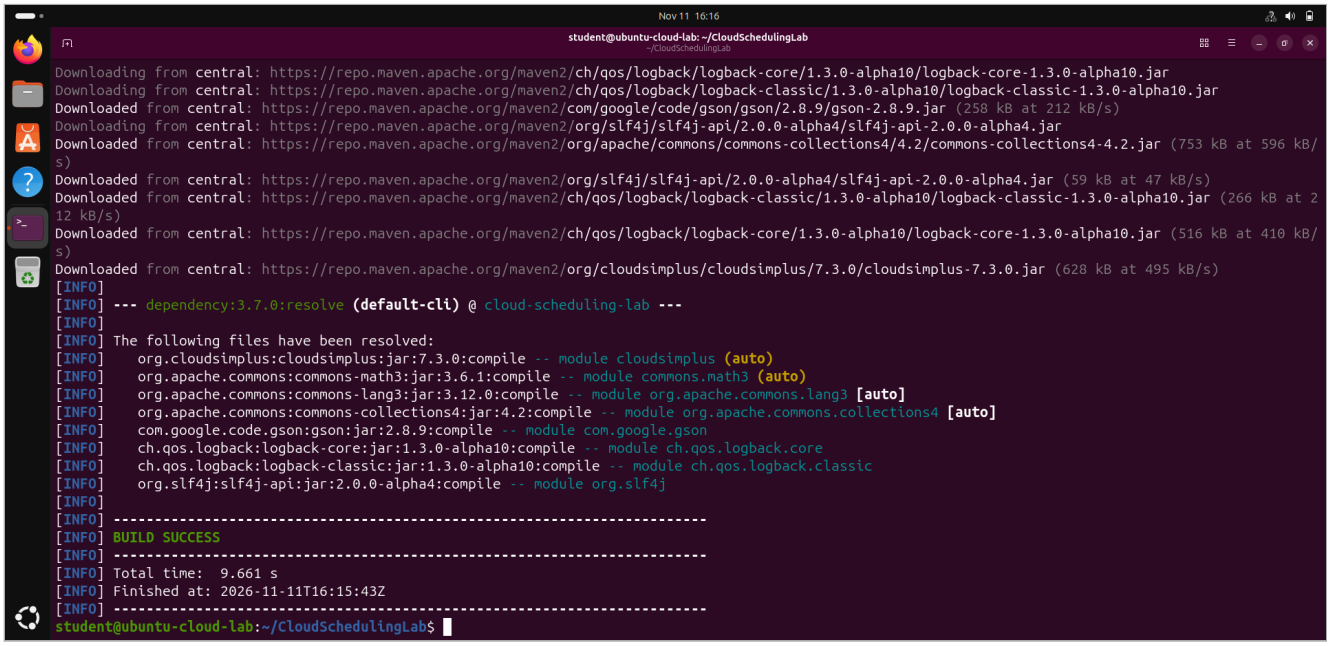


Figure 23: `mvn dependency:resolve` with `cloudsimplus:jar:7.3.0`

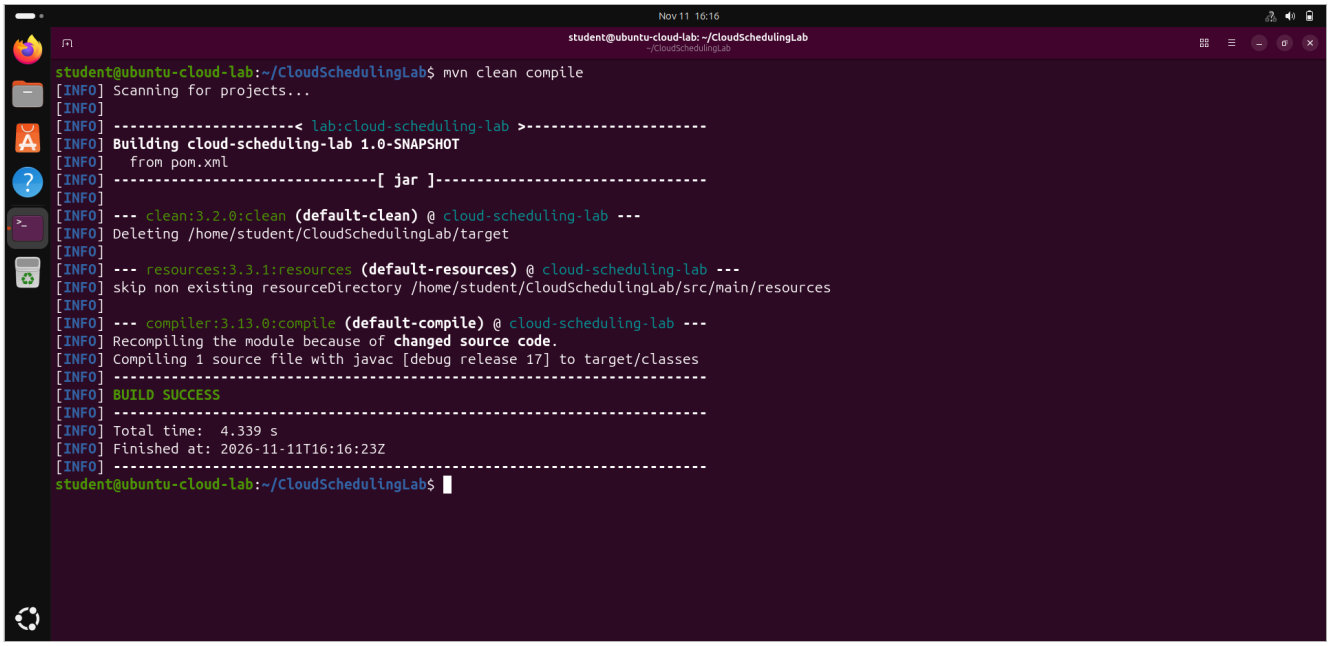


Figure 24: `mvn clean compile` with 7.3.0: BUILD SUCCESS

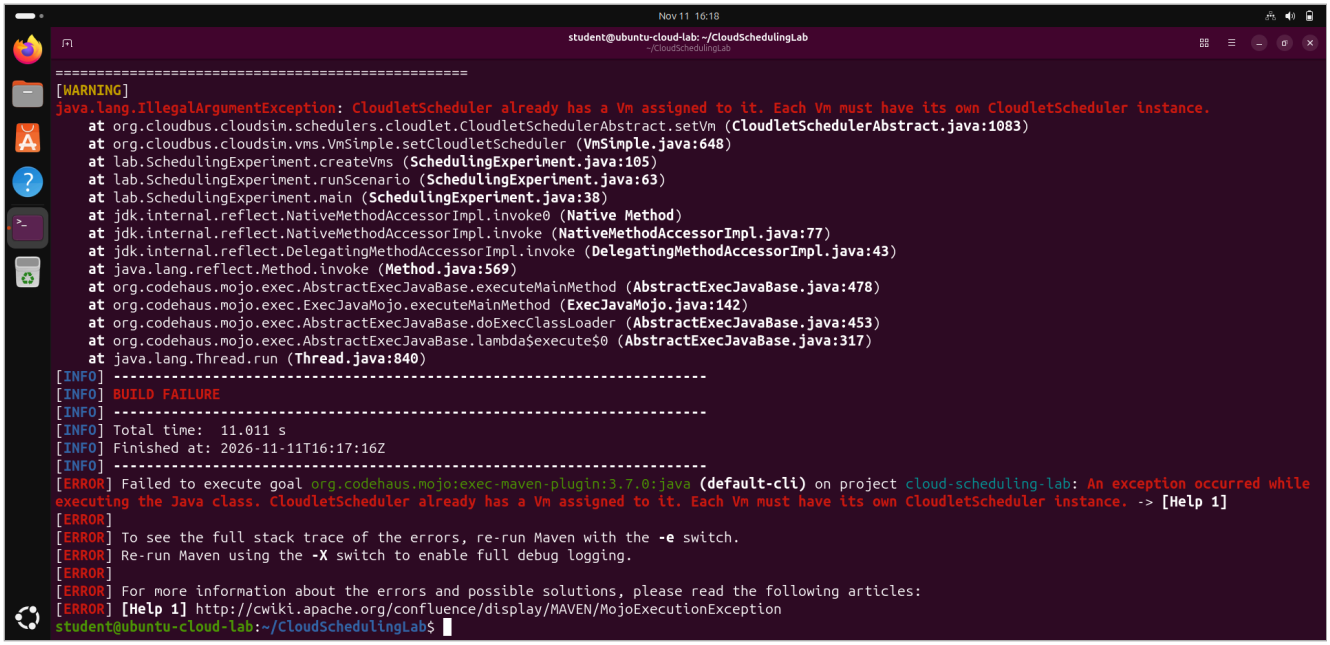


Figure 25: `mvn exec:java` fails: `CloudletScheduler already has a Vm assigned to it`

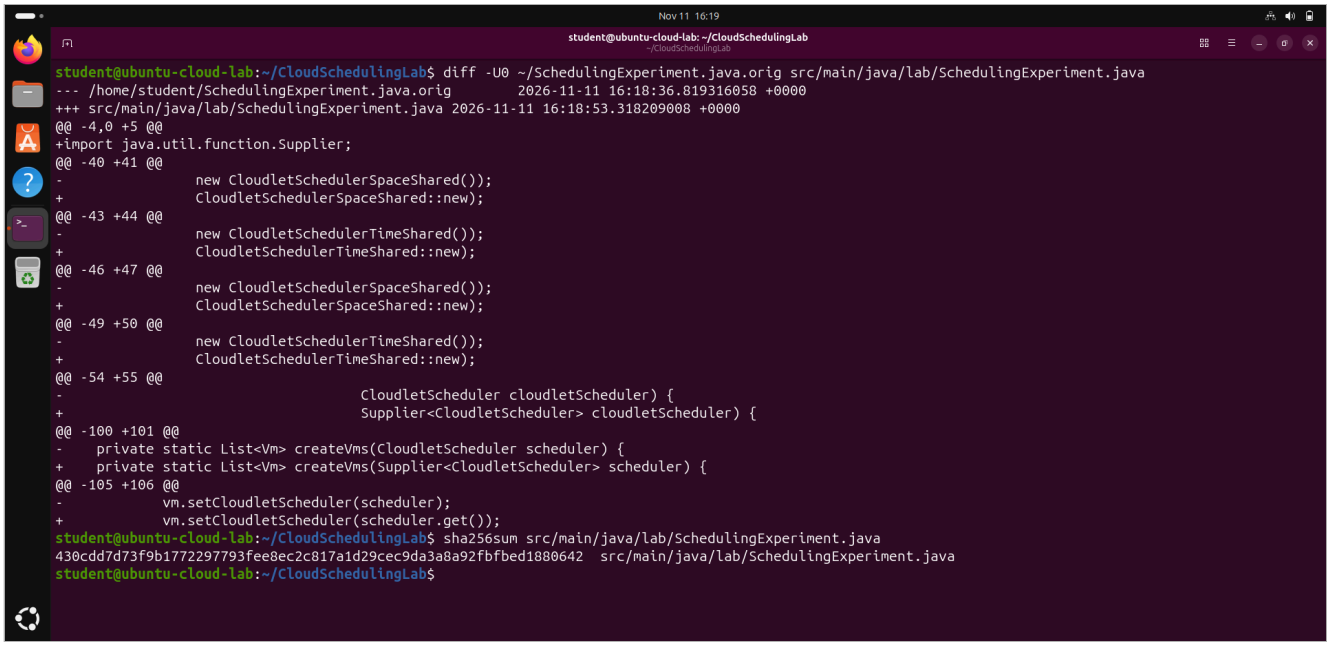


Figure 26: Fix 2: diff that creates one scheduler per VM

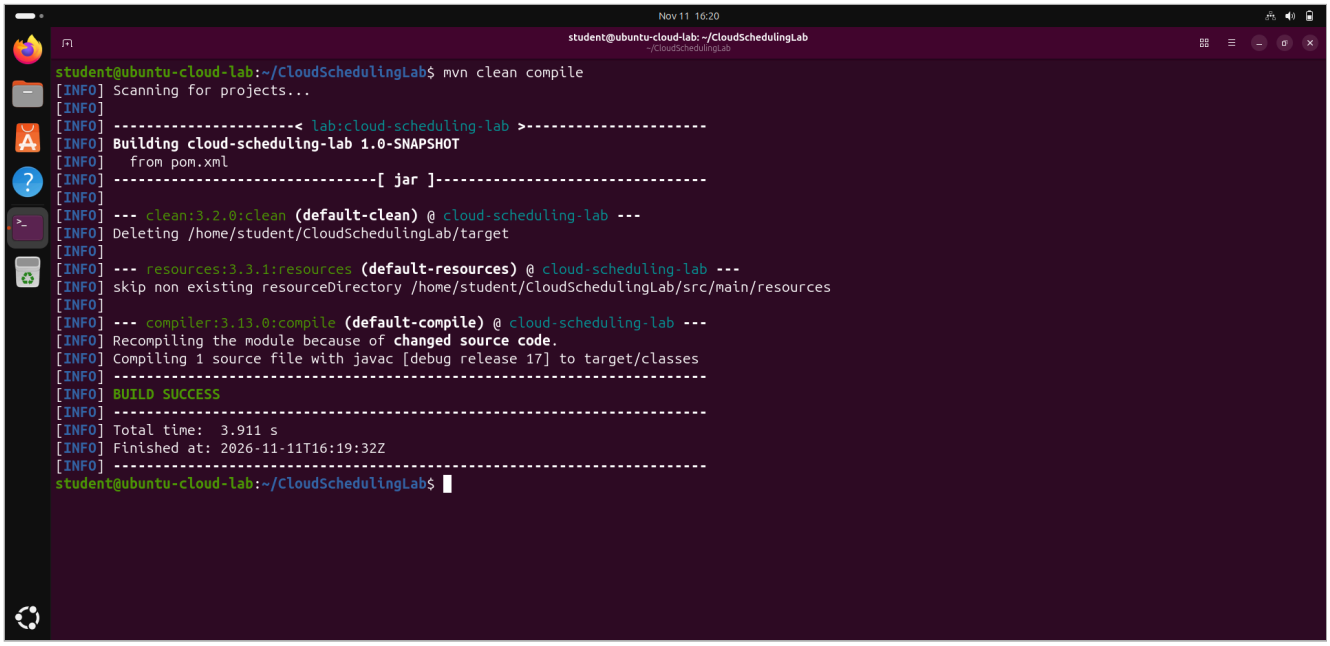


Figure 27: `mvn clean compile` after fix 2: BUILD SUCCESS

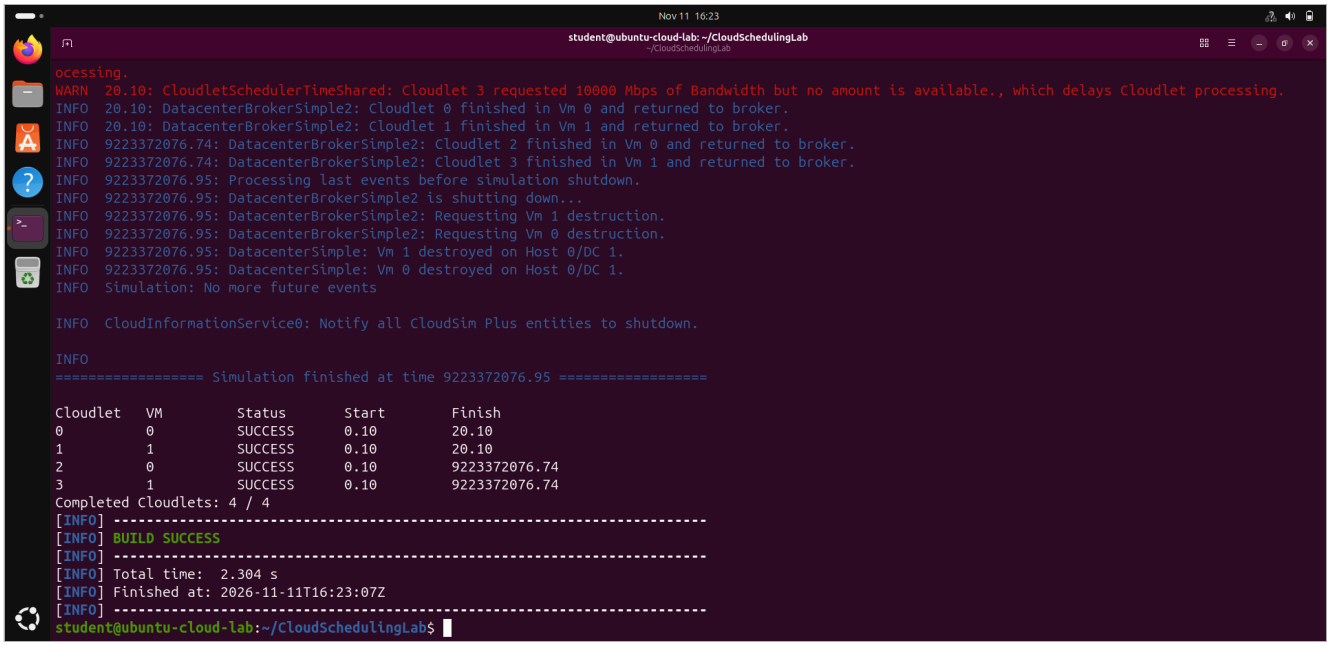


Figure 28: Run 1: `mvn exec:java` output, BUILD SUCCESS but finish time 9223372076.74

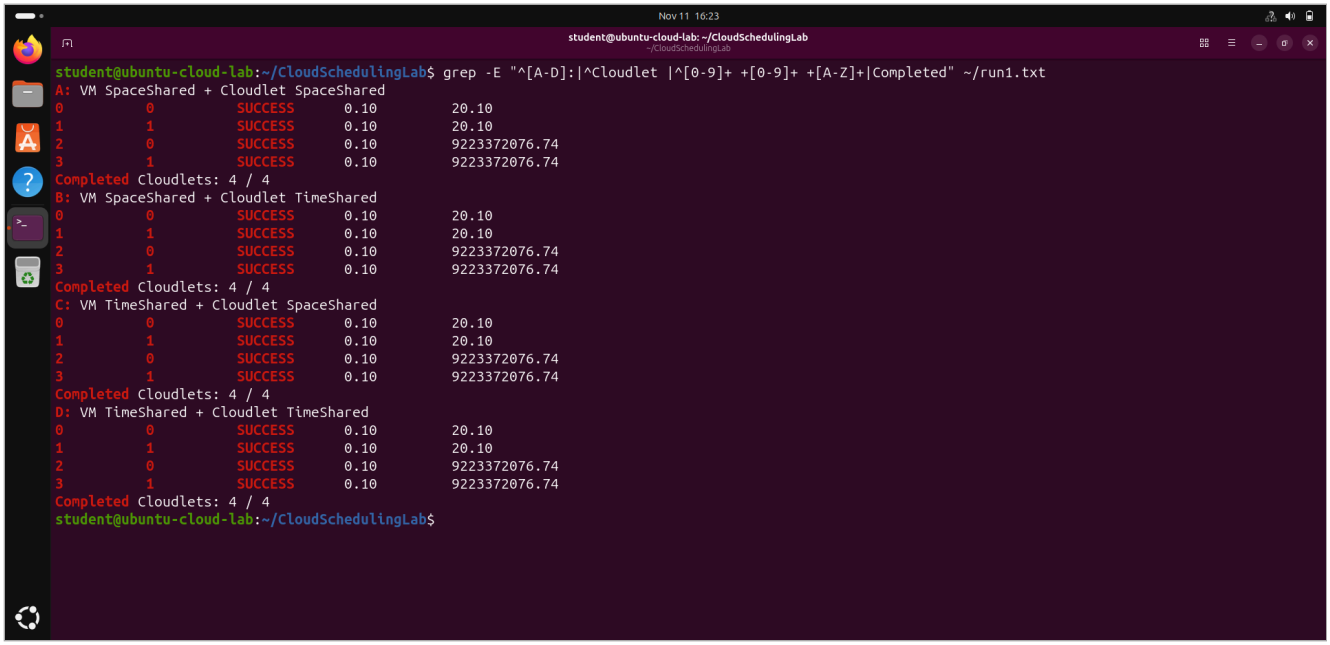


Figure 29: Run 1: summary of scenarios A–D

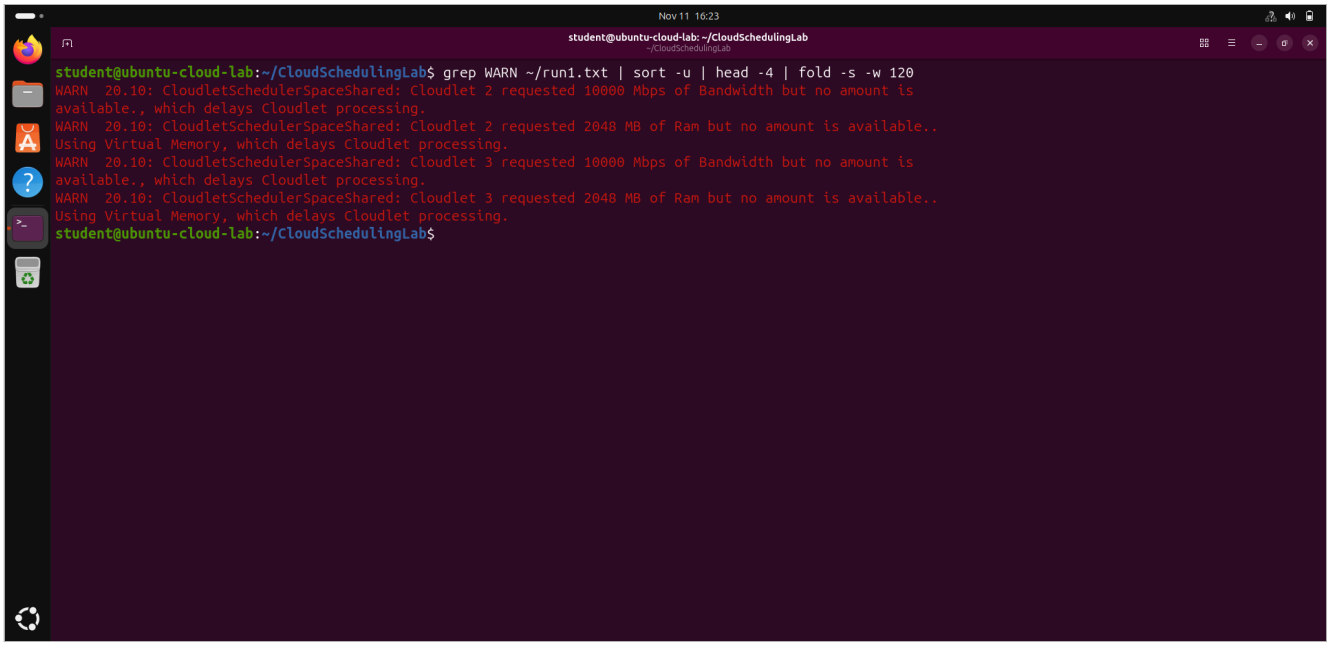


Figure 30: Run 1: warnings about RAM and bandwidth

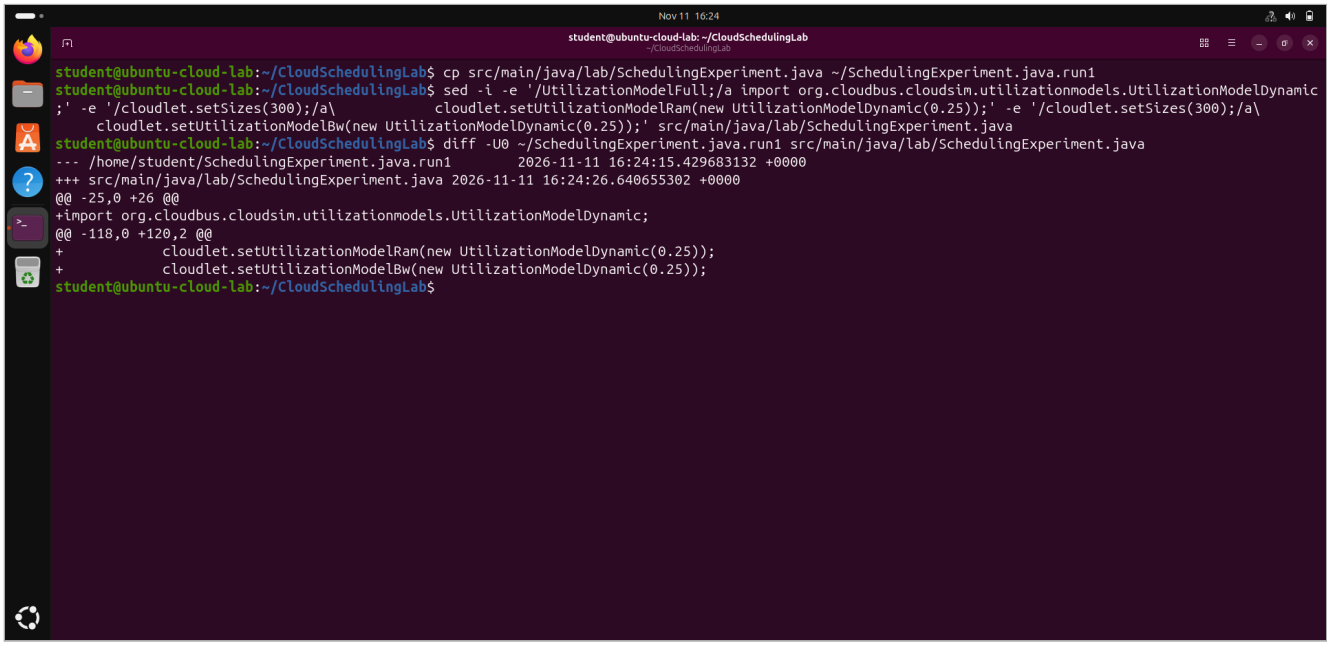


Figure 31: Fix 3: diff that sets cloudlet RAM and bandwidth utilization to 25%

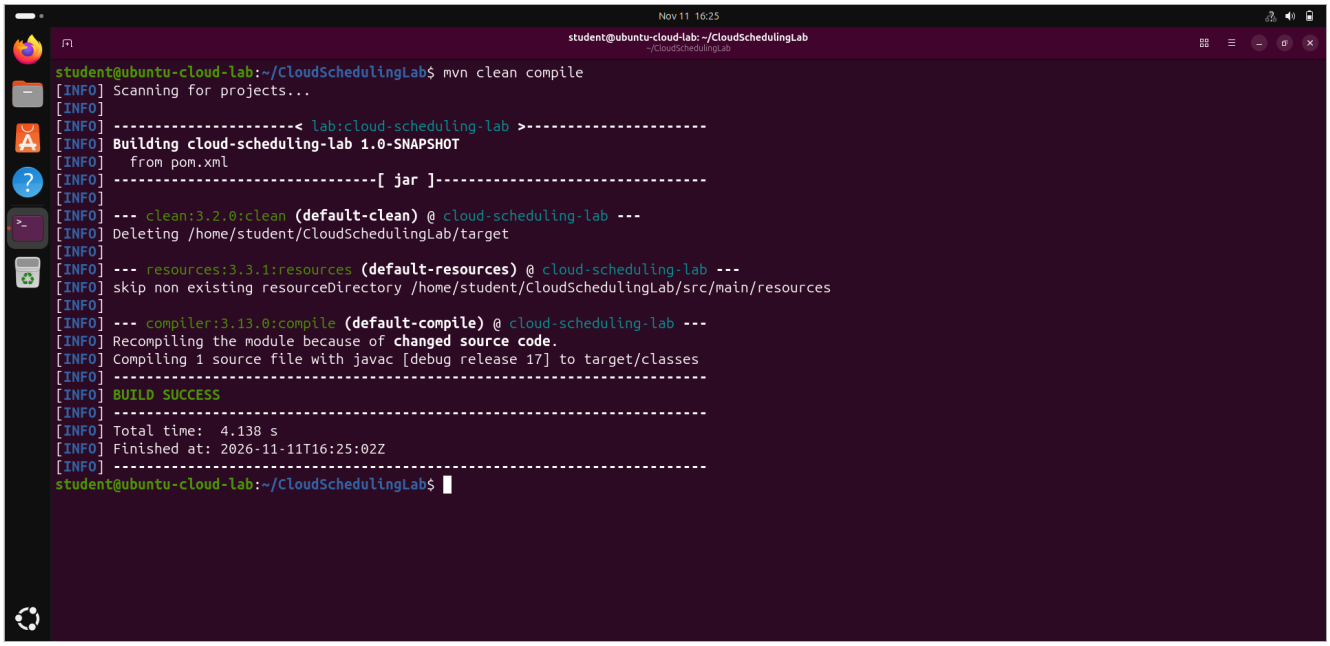


Figure 32: Run 2: `mvn clean compile`: BUILD SUCCESS

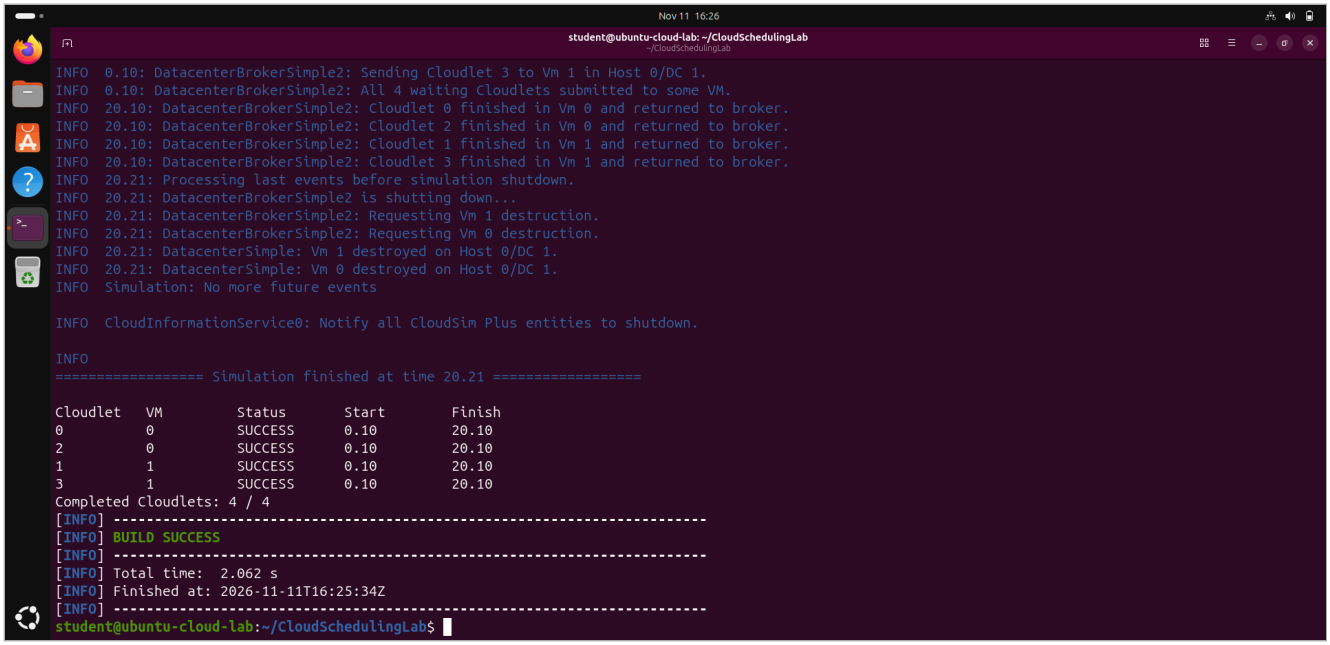


Figure 33: Run 2: `mvn exec:java` output, all cloudlets 0.10 → 20.10, 4 / 4

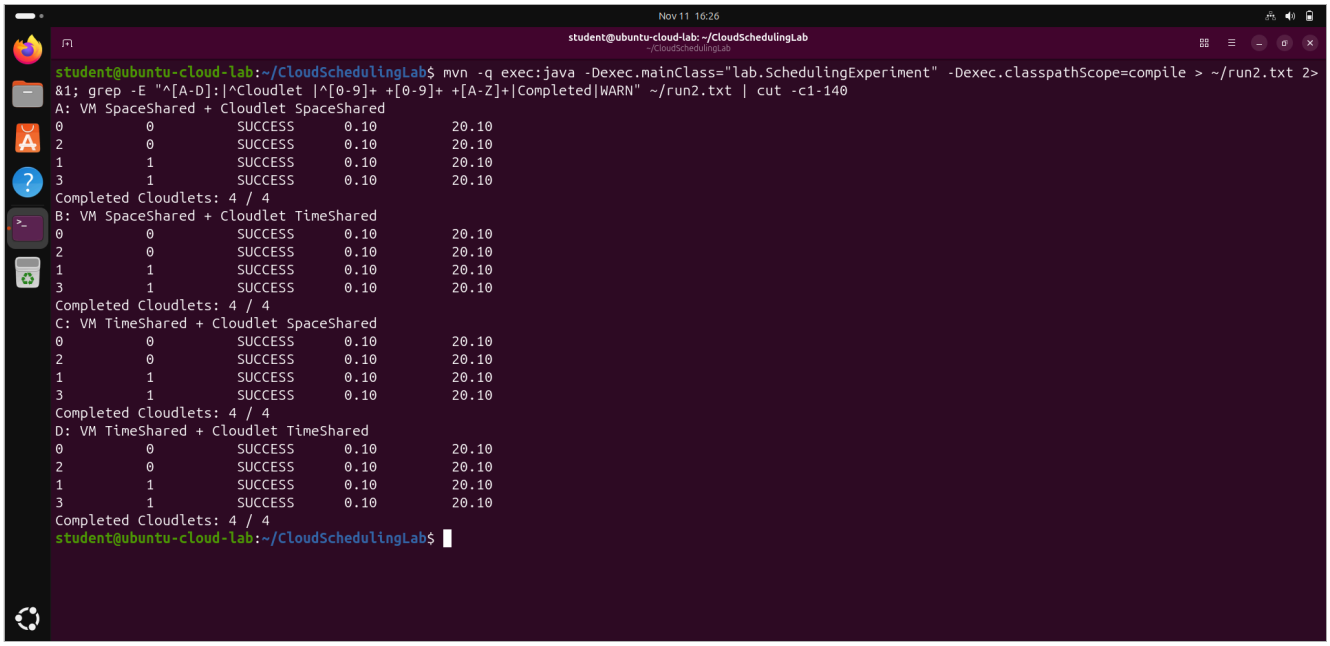


Figure 34: Run 2: summary of scenarios A–D

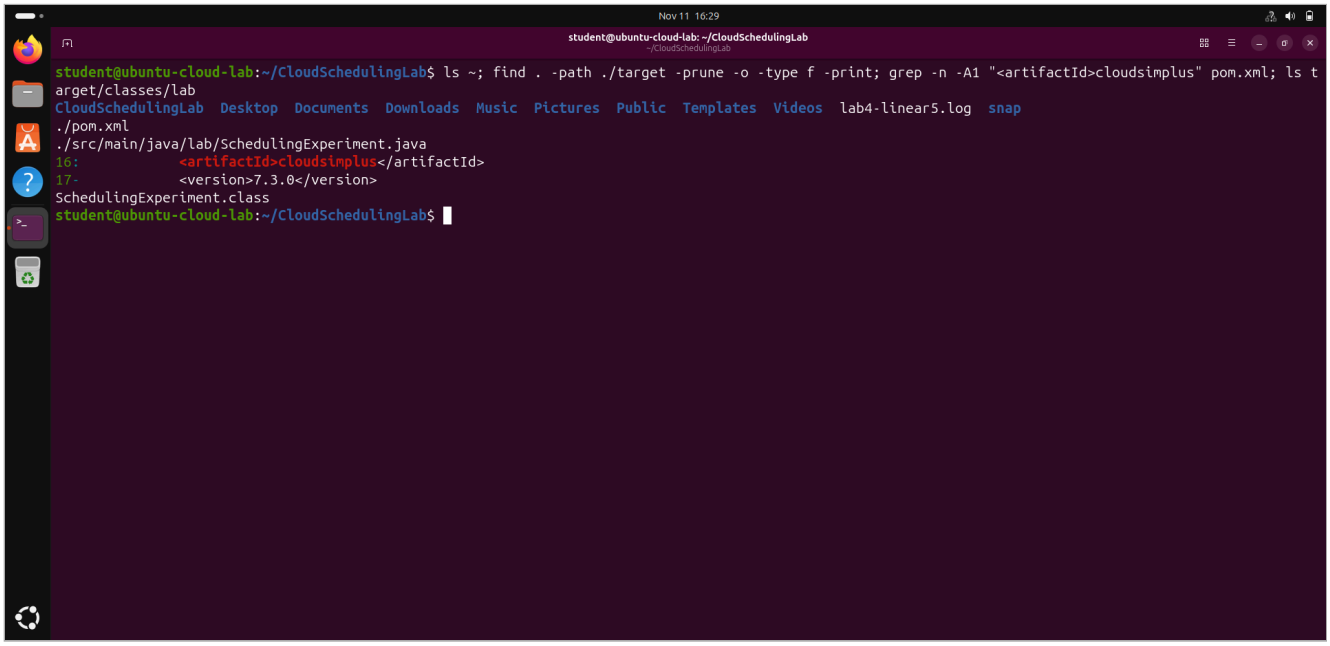


Figure 35: Final project layout and pinned dependency version

8. Result

Java 17.0.21, Maven 3.9.12 and Git 2.53.0 were installed on Ubuntu 26.04.1 and a CloudSim Plus project was created. After the fixes in section 5, the program compiled (`BUILD SUCCESS`, `javac release 17`) and simulated all four scheduling combinations. In each of scenarios A, B, C and D, **4 / 4 cloudlets completed with status SUCCESS**, starting at 0.10 s and finishing at 20.10 s (20 s of work for 20000 MI at 1000 MIPS plus the 0.1 s start-up delay).

9. Analysis and conclusion

Why all four scenarios look the same. This setup never forces a policy to make a difference:

- *VM level:* the host has 4 PEs and the two VMs need 2 PEs each (2 + 2 = 4). Nothing is oversubscribed, so space-shared and time-shared hosts both give every VM its own full PEs.
- *Cloudlet level:* each VM has 2 PEs and receives 2 one-PE cloudlets, so even the space-shared scheduler can run both at once on separate PEs, and the time-shared scheduler has nothing to divide.

When they would differ (expected behaviour, not run here). If a VM received more cloudlets than it has PEs (for example 4 cloudlets on a 2-PE VM), a space-shared cloudlet scheduler would run 2 first (finishing at 20.10) and queue the other 2 (finishing at 40.10), while a time-shared scheduler would run all 4 together at half speed (all finishing at 40.10). In the same way, a time-shared VM scheduler would let VMs be oversubscribed on a host, while a space-shared one would refuse to place a VM that does not fit.

What the experiment showed about the tool. Run 1 against Run 2 shows that results from a simulator depend on the whole model, not only the scheduler being studied: asking every cloudlet for 100% of the VM's RAM and bandwidth serialised them on each VM, whatever the CPU scheduling policy was. The manual's code also needed three corrections before it ran, because it was written for a different CloudSim Plus version than the one named in its `pom.xml`. Keeping the original files, the patches and a screenshot of every failure makes each change traceable.